

Intro to Annotation

Michael Schatz

October 20, 2024

Lecture 16. Applied Comparative Genomics



Project Proposal

Due Wednesday Oct 15 @ 11:59pm

Project Proposal

Assignment Date: Monday October 6, 2025

Due Date: Wednesday, October 15 2025 @ 11:59pm

Review the [Project Ideas](#) page

Work solo or form a team for your class project of no more than 3 people.

The proposal should have the following components:

- Name of your team
- List of team members and email addresses
- Short title for your proposal
- 1 to 2 paragraph description of what you hope to do and how you will do it
- References to 3 to 4 relevant papers
- References/URLs to datasets that you will be studying (Note you can also use simulated data)
- Please add a note if you need me to sponsor you for a MARCC account (high RAM, GPUs, many cores, etc)
- Please add a note if you need me to sponsor you for an AnVIL cloud billing account (dynamic resources)

Submit the proposal as a 1 to 2 page PDF on GradeScope (each team should submit one proposal and tag all people in the team). After submitting your proposal, I will provide feedback. If necessary, we can schedule a time to discuss your proposal, especially to ensure you have access to the data that you need. The sooner that you submit your proposal, the sooner we can schedule the meeting.

Later, you will present your project in class during the last week of class. You will also submit a written report (5-7 pages) of your project, formatting as a Bioinformatics article (Intro, Methods, Results, Discussion, References). Word and LaTeX templates are available at https://academic.oup.com/bioinformatics/pages/submission_online

Please use Piazza to coordinate proposal plans!

Assignment 4

Due Wednesday Oct 22 @ 11:59pm

Assignment 4: RNA-seq and Machine Learning

Assignment Date: Wednesday, October 15, 2025

Due Date: Wednesday, October 22 @ 11:59pm

Assignment Overview

In this assignment you will explore a couple of key aspects of RNA-seq and introduce the key concepts of machine learning. For this assignment, we will provide a Jupyter notebook with code for you to use and complete your assignment in.

To use the Jupyter notebook, create a new mamba environment for assignment 4:

```
mamba create -n asn4 scikit-learn pytorch matplotlib jupyter umap-learn seaborn pandas numpy
```

Make sure to download the training data for the assignment from [Dropbox](#). Place this in the `data_files` directory and make sure to uncompress (`gunzip`) both the training and testing files.

As a reminder, any questions about the assignment should be posted to [Piazza](#).

See the notebook here: [Assignment4.ipynb](#)

Packaging

The solutions to the above questions should be submitted as a single PDF document that includes your name, email address, and all relevant figures (as needed). If you use ChatGPT for any of the code, also record the prompts used. Submit your solutions by uploading the PDF to [GradeScope](#), and remember to select where in your submission each question/subquestion is. The Entry Code is: GVXGV2.

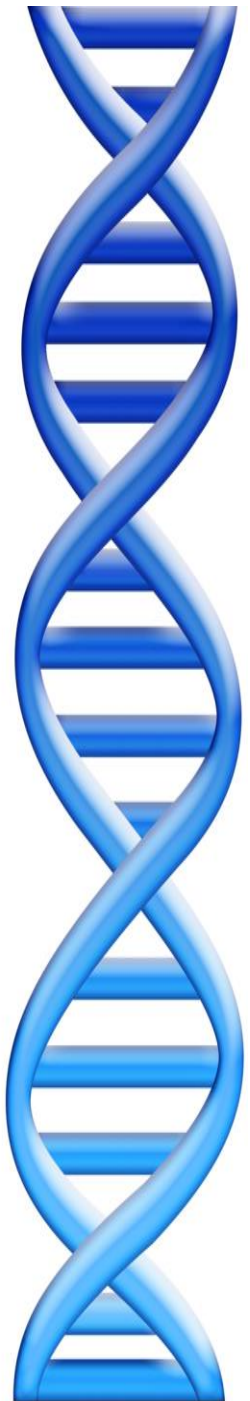
If you submit after this time, you will use your late days. Remember, you are only allowed 4 late days for the entire semester!

Resources

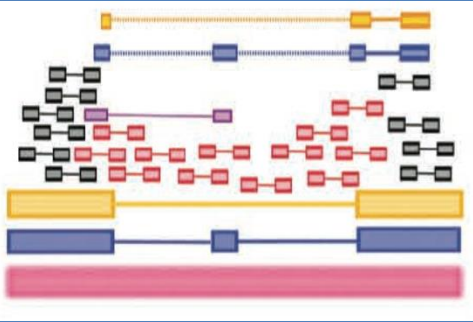
- Jupyter notebooks: <https://jupyter.org/>
- scikit-learn: <https://scikit-learn.org/stable/>
- pytorch: <https://pytorch.org/>
- umap-learn: https://umap-learn.readthedocs.io/en/latest/basic_usage.html

Outline

1. Alignment to other genomes
2. Prediction aka “Gene Finding”
3. Experimental & Functional Assays



RNA-seq Challenges

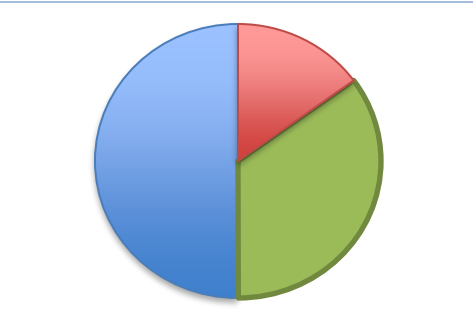


Challenge 1: Eukaryotic genes are spliced

Solution: Use a spliced aligner, and assemble isoforms

TopHat: discovering spliced junctions with RNA-Seq.

Trapnell et al (2009) *Bioinformatics*. 25:0 1105-1111

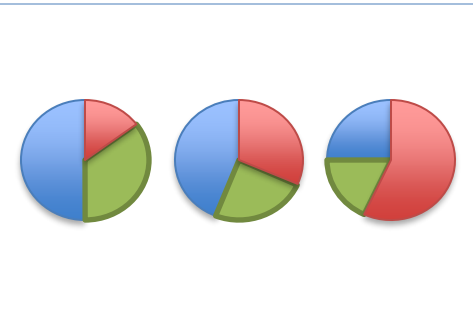


Challenge 2: Read Count != Transcript abundance

Solution: Infer underlying abundances (e.g. TPM)

Transcript assembly and quantification by RNA-seq

Trapnell et al (2010) *Nat. Biotech.* 25(5): 511-515



Challenge 3: Transcript abundances are stochastic

Solution: Replicates, replicates, and more replicates

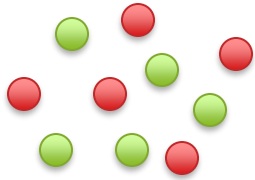
RNA-seq differential expression studies: more sequence or more replication?

Liu et al (2013) *Bioinformatics*. doi:10.1093/bioinformatics/btt688

Population Heterogeneity

Red cells express twice the abundance of “brain” genes compared to green cells

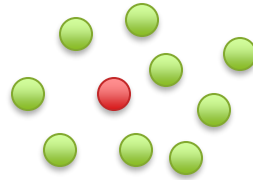
Experiment 1: 50/50



Compared to a control sample of pure green cells, this sample will show:

$50\% 2x + 50\% 1x$
= 1.5x over expression of brain genes

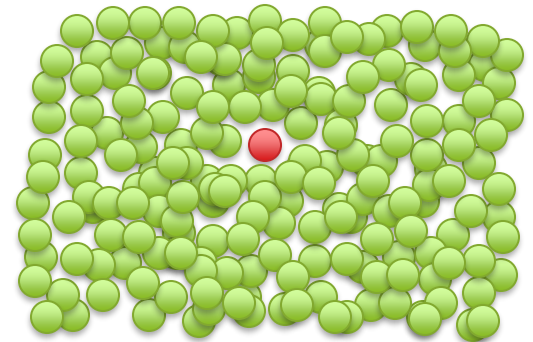
Experiment 2: 1/10



Compared to a control sample of pure green cells, this sample will show:

$10\% 2x + 90\% 1x$
= 1.1x over expression of brain genes

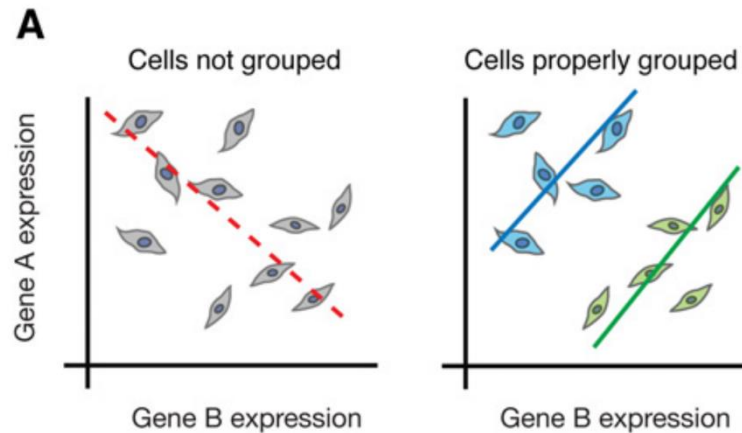
Experiment 3: 1/1000



Compared to a control sample of pure green cells, this sample will show:

$0.1\% 2x + 99.1\% 1x$
= 1.001x over expression of brain genes

The paradox of averages



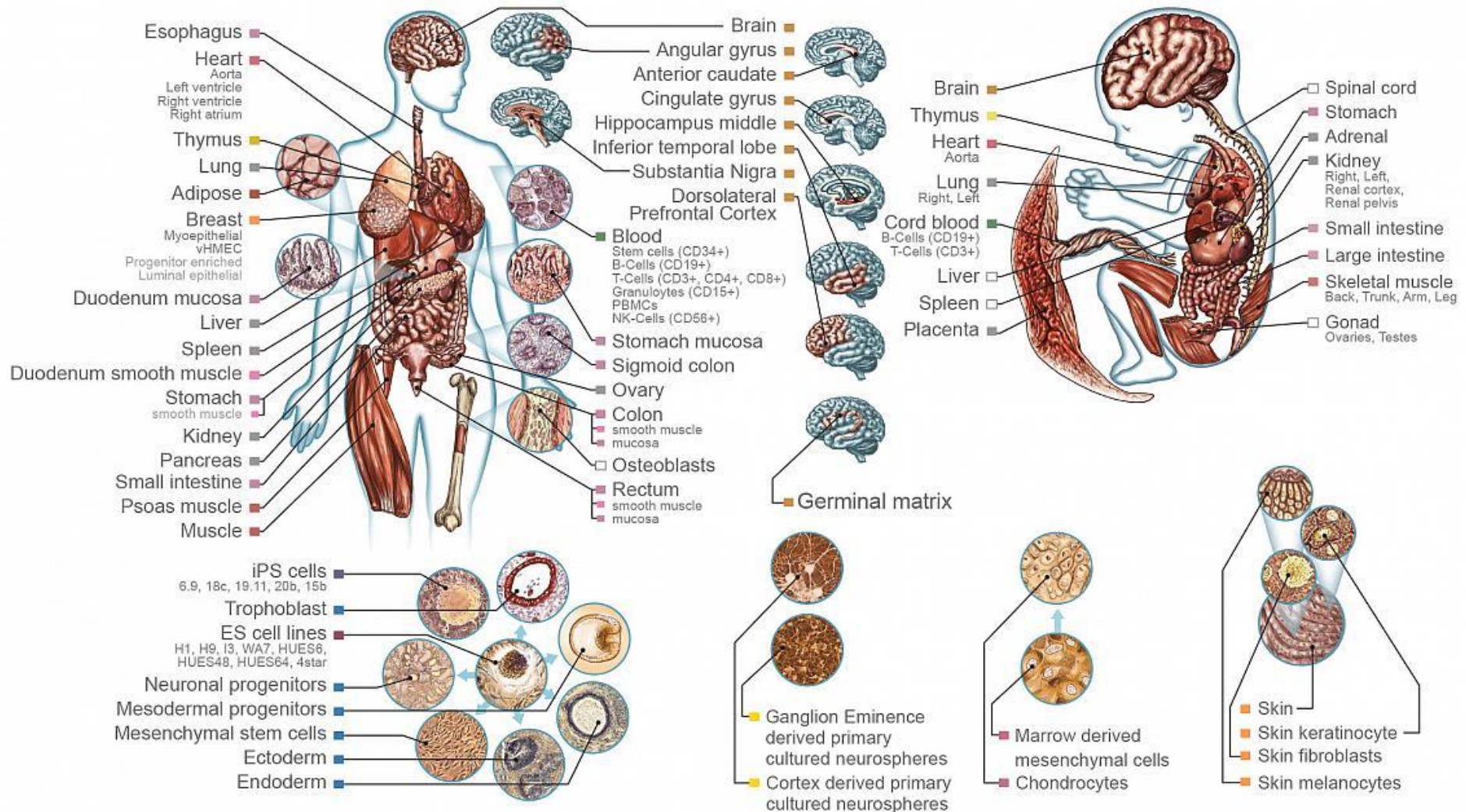
What??? How can the better performing drug depend on if you examine the overall response or separately examine by gender

Example of Simpson's paradox:

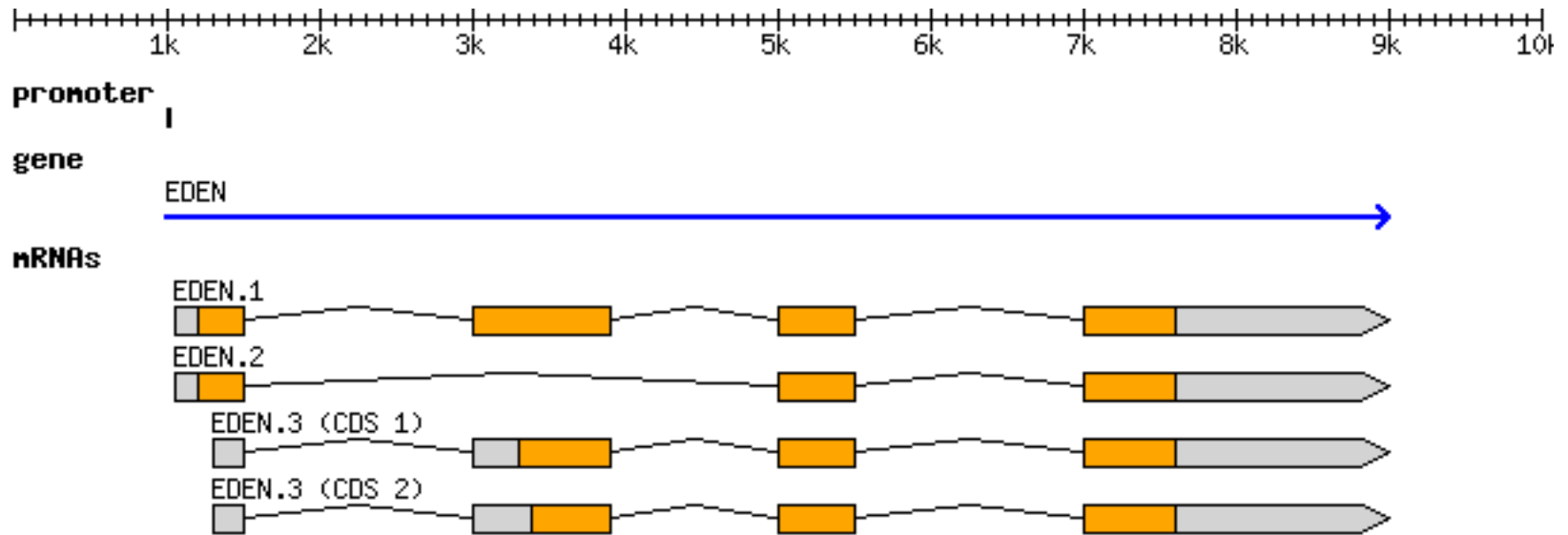
Trend of the overall average may reverse the trends of each constituent group

In this example, the "lurking" variable (or confounding variable) is the severity of the case (represented by the doctors' treatment decision trend of favoring B for less severe cases), which was not previously known to be important until its effects were included. (Based on real analysis of kidney stone treatments)

Sources of (Cellular) Heterogeneity



Gene Models



- “Generic Feature Format” (GFF) records genomic features
 - Coordinates of each exon
 - Coordinates of UTRs
 - Link together exons into transcripts
 - Link together transcripts into gene models

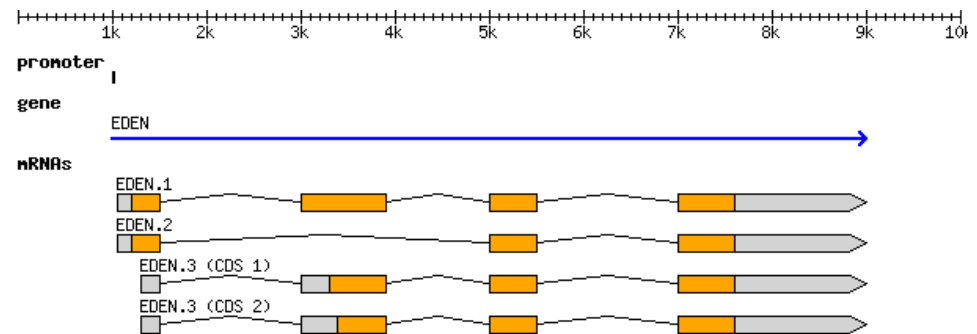
GFF File format

GFF3 files are nine-column, tab-delimited, plain text files

- 1. *seqid*:** The ID of the sequence
- 2. *source*:** Algorithm or database that generated this feature
- 3. *type*:** *gene/exon/CDS/etc...*
- 4. *start*:** 1-based coordinate
- 5. *end*:** 1-based coordinate
- 6. *score*:** E-values/p-values/index/colors/...
- 7. *strand*:** “+” for positive “-” for minus, “.” not stranded
- 8. *phase*:** For "CDS", where the feature begins with reference to the reading frame (0,1,2)
- 9. *attributes*:** A list of tag=value features
Parent: Indicates the parent of the feature (group exons into transcripts, transcripts into genes, ...)

GFF Example

Gene “EDEN” with 3 alternatively spliced transcripts, isoform 3 has two alternative translation start sites



```
##gff-version 3
##sequence-region ctg123 1 1497228
ctg123 . gene 1000 9000 . + . ID=gene00001;Name=EDEN

ctg123 . TF_binding_site 1000 1012 . + . ID=tfbs00001;Parent=gene00001

ctg123 . mRNA 1050 9000 . + . ID=mRNA00001;Parent=gene00001;Name=EDEN.1
ctg123 . mRNA 1050 9000 . + . ID=mRNA00002;Parent=gene00001;Name=EDEN.2
ctg123 . mRNA 1300 9000 . + . ID=mRNA00003;Parent=gene00001;Name=EDEN.3

ctg123 . exon 1300 1500 . + . ID=exon00001;Parent=mRNA00003
ctg123 . exon 1050 1500 . + . ID=exon00002;Parent=mRNA00001,mRNA00002
ctg123 . exon 3000 3902 . + . ID=exon00003;Parent=mRNA00001,mRNA00003
ctg123 . exon 5000 5500 . + . ID=exon00004;Parent=mRNA00001,mRNA00002,mRNA00003
ctg123 . exon 7000 9000 . + . ID=exon00005;Parent=mRNA00001,mRNA00002,mRNA00003

ctg123 . CDS 1201 1500 . + 0 ID=cds00001;Parent=mRNA00001;Name=edenprotein.1
ctg123 . CDS 3000 3902 . + 0 ID=cds00001;Parent=mRNA00001;Name=edenprotein.1
ctg123 . CDS 5000 5500 . + 0 ID=cds00001;Parent=mRNA00001;Name=edenprotein.1
ctg123 . CDS 7000 7600 . + 0 ID=cds00001;Parent=mRNA00001;Name=edenprotein.1

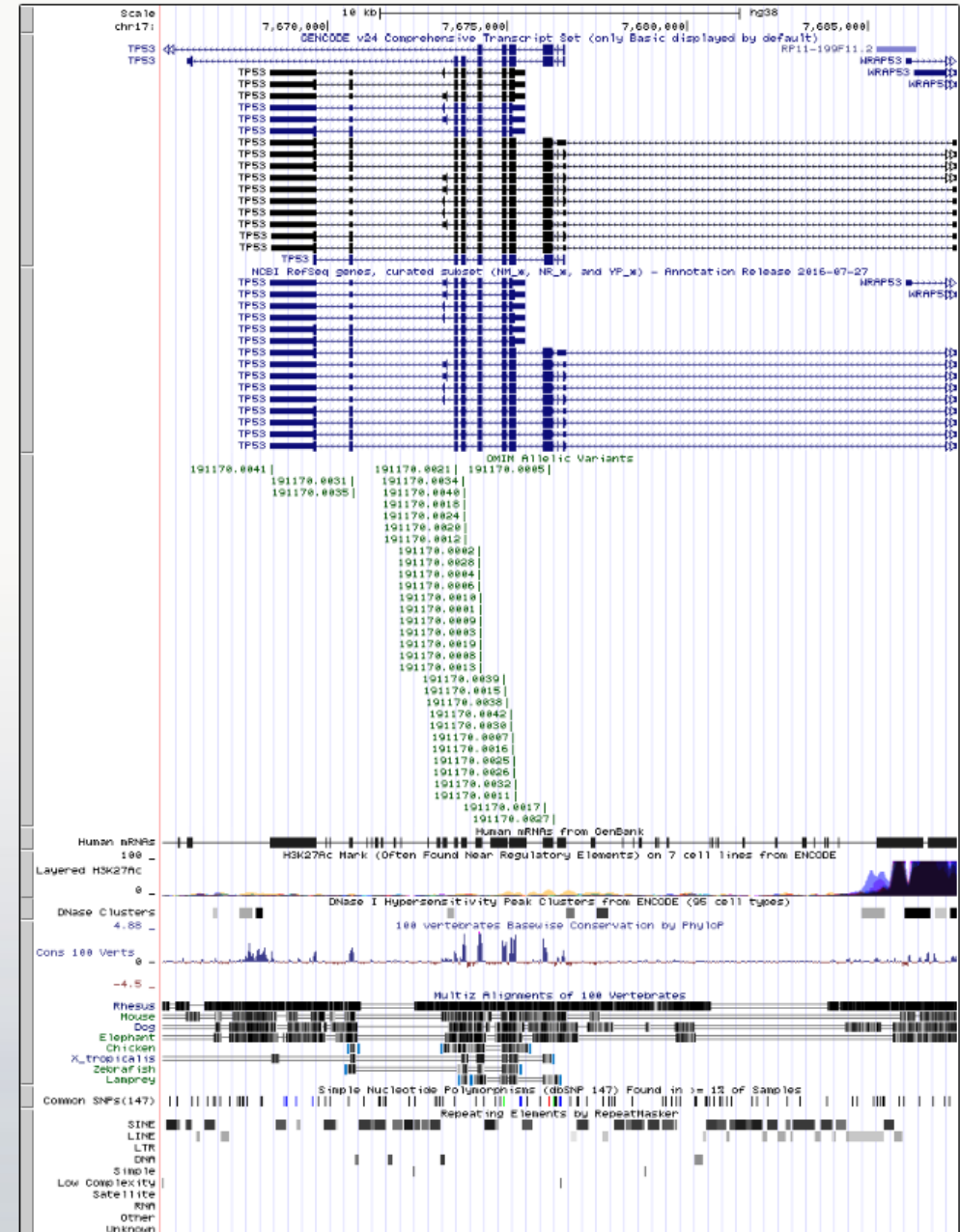
ctg123 . CDS 1201 1500 . + 0 ID=cds00002;Parent=mRNA00002;Name=edenprotein.2
ctg123 . CDS 5000 5500 . + 0 ID=cds00002;Parent=mRNA00002;Name=edenprotein.2
ctg123 . CDS 7000 7600 . + 0 ID=cds00002;Parent=mRNA00002;Name=edenprotein.2

ctg123 . CDS 3301 3902 . + 0 ID=cds00003;Parent=mRNA00003;Name=edenprotein.3
ctg123 . CDS 5000 5500 . + 1 ID=cds00003;Parent=mRNA00003;Name=edenprotein.3
ctg123 . CDS 7000 7600 . + 1 ID=cds00003;Parent=mRNA00003;Name=edenprotein.3

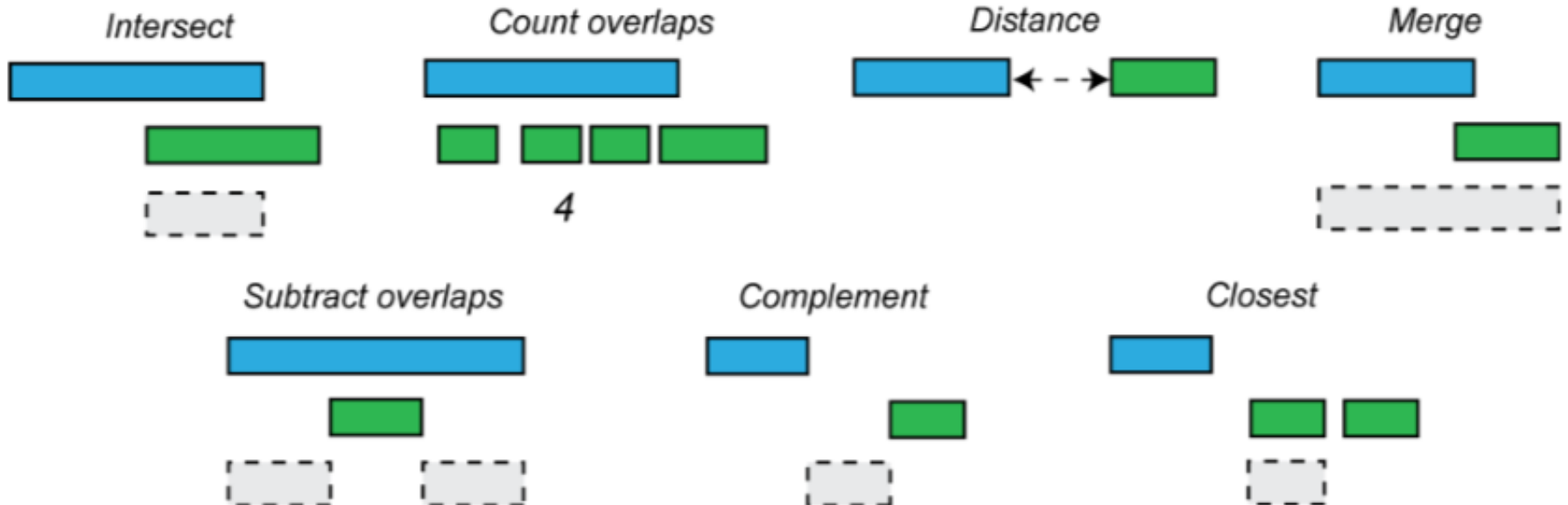
ctg123 . CDS 3391 3902 . + 0 ID=cds00004;Parent=mRNA00003;Name=edenprotein.4
ctg123 . CDS 5000 5500 . + 1 ID=cds00004;Parent=mRNA00003;Name=edenprotein.4
ctg123 . CDS 7000 7600 . + 1 ID=cds00004;Parent=mRNA00003;Name=edenprotein.4
```

UCSC Genome Browser on Human Dec. 2013 (GRCh38/hg38) Assembly

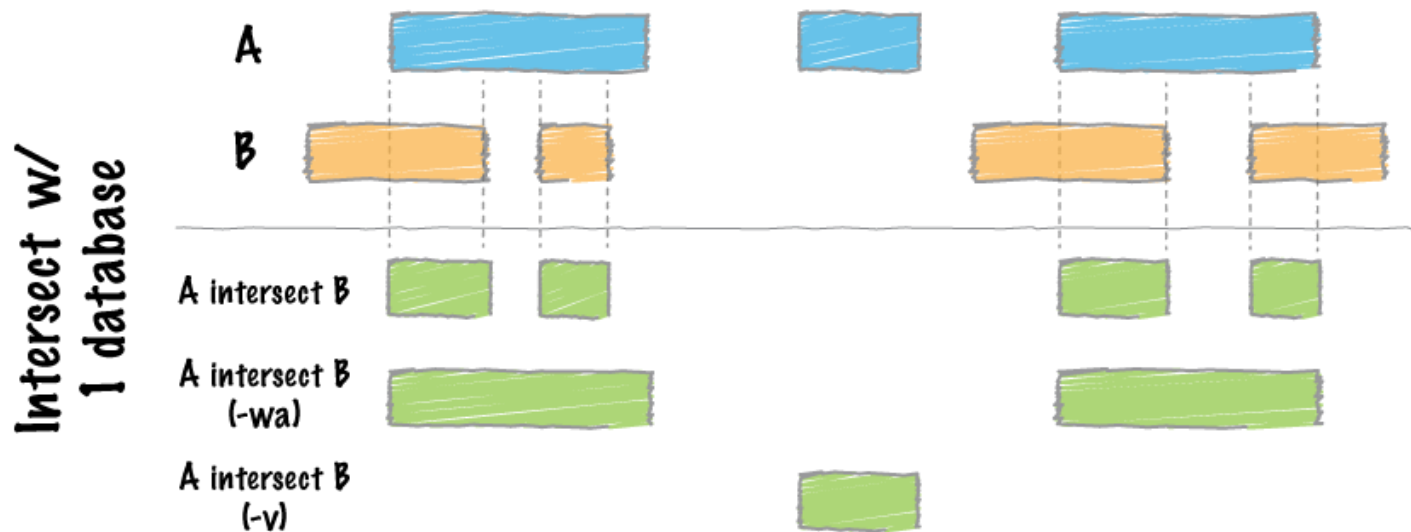
- move <<< << >> >>> zoom in 1.5x 3x 10x base zoom out 1.5x 3x 10x 100x
- chr17:7,665,416-7,687,491 22,076 bp. enter position, gene symbol or search terms go
- chr17 (p13.1) 17p11.2 17p11.2 g11.2 17q12 17q22 24.525.1 24.525.3
- Scale chr17: 10 kb hg38 7,670,000 7,675,000 7,680,000 7,685,000
- TPSS v24 Comprehensive Transcript Set (only Basic displayed by default)
- TPSS
- NCBI RefSeq genes, curated subset (NM_x, NR_x, and YP_x) - Annotation Release 2015-07-27
- TPSS
- OMIM Allelic Variants
- Human mRNAs from GenBank
- H3K27Ac Mark (Often Found Near Regulatory Elements) on 7 cell lines from ENCODE
- DNAse I Hypersensitivity Peak Clusters from ENCODE (96 cell types)
- Cons 100 Verts
- Multiz Alignments of 100 Vertebrates
- Simple Nucleotide Polymorphisms (dbSNP 147) Found in ≥ 1% of Samples
- Repeating Elements by RepeatMasker
- SINE LINE LTR DNA SINE Other Unknown
- Low Complexity Satellite RNA Other Unknown
- Click on a feature for details. Click or drag in the base position track to zoom.



BEDTools to the rescue!



BEDTools Intersect



What exons are hit by SVs?

```
$ cat A.bed  
chr1 10 20  
chr1 30 40
```

```
$ cat B.bed  
chr1 15 20
```

```
$ bedtools intersect -a A.bed -b B.bed -wa  
chr1 10 20
```

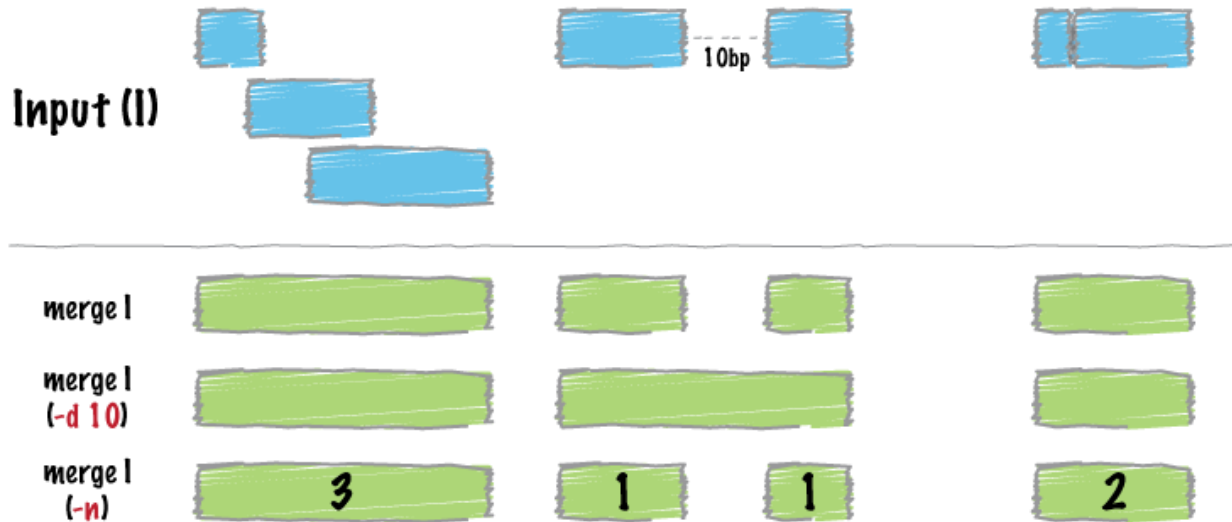
What parts of exons are hit by SVs?

```
$ cat A.bed  
chr1 10 20  
chr1 30 40
```

```
$ cat B.bed  
chr1 15 20
```

```
$ bedtools intersect -a A.bed -b B.bed  
chr1 15 20
```


BEDTools Merge



What parts of the genome are exonic?

```
bedtools merge -i exons.bed | head -n 20
```

chr1	11873	12227
chr1	12612	12721
chr1	13220	14829
chr1	14969	15038
chr1	15795	15947
chr1	16606	16765
chr1	16857	17055
chr1	17333	17360

Note input must be sorted!

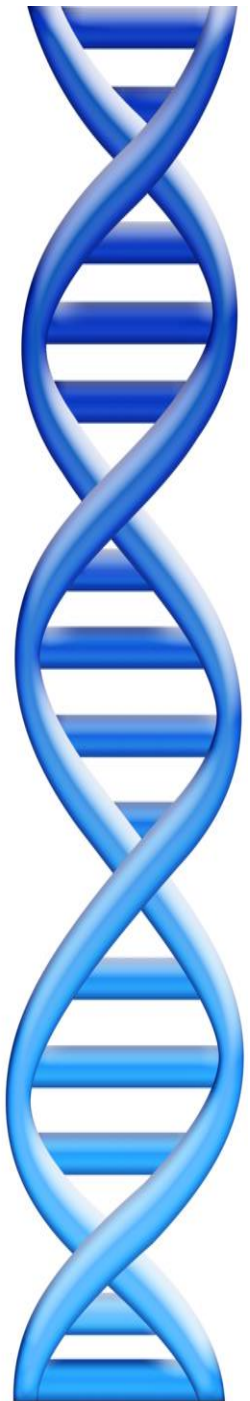
```
sort -k1,1 -k2,2n foo.bed > foo.sort.bed
```

BEDTools commands

annotate	getfasta	overlap
bamtobed	groupby	pairtobed
bamtofastq	groupby	pairtopair
bed12tobed6	igv	random
bedpetobam	intersect	reldist
bedtobam	jaccard	shift
closest	links	shuffle
cluster	makewindows	slop
complement	map	sort
coverage	maskfasta	subtract
expand	merge	tag
flank	multicov	unionbedg
fisher	multiinter	window
genomecov	nuc	

Outline

1. Alignment to other genomes
2. Prediction aka “Gene Finding”
3. Experimental & Functional Assays



Basic Local Alignment Search Tool

- Rapidly compare a sequence Q to a database to find all sequences in the database with a score above some cutoff S .
 - Which protein is most similar to a newly sequenced one?
 - Where does this sequence of DNA originate?
- Speed achieved by using a procedure that typically finds “most” matches with scores $> S$.
 - Tradeoff between sensitivity and specificity/speed
 - Sensitivity – ability to find all related sequences
 - Specificity – ability to reject unrelated sequences

Seed and Extend

FAKDFLAGGVAAAI SKTAVAPI IERVKLLQVQHASKQITADKQYKGI IDCVVRI PKEQGV
FLIDLASGGTAAAV SKTAVAPI IERVKLLQVQDASKAIAVDKRYKGIMDVLIRVPKEQGV

- Homologous sequences are likely to contain a **short high scoring word pair**, a seed.
 - Smaller seed sizes make the sense more sensitive, but also (much) slower
 - Typically do a fast search for prototypes, but then most sensitive for final result
- BLAST then tries to extend high scoring word pairs to compute **high scoring segment pairs** (HSPs).
 - Significance of the alignment reported via an e-value

Seed and Extend

```
FAKDFLAGGVAAAI SKTAVAPIERVKLLQLQVQHASKQITADKQYKGIIDCVVRIPKEQGV
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
FLIDLASGGTAAAVS SKTAVAPIERVKLLQLQVQDASKAIAVDKRYKGIMDVLIRVPKEQGV
```

- Homologous sequences are likely to contain a **short high scoring word pair**, a seed.
 - Smaller seed sizes make the sense more sensitive, but also (much) slower
 - Typically do a fast search for prototypes, but then most sensitive for final result
- BLAST then tries to extend high scoring word pairs to compute **high scoring segment pairs** (HSPs).
 - Significance of the alignment reported via an e-value

BLAST E-values

E-value = the number of HSPs having alignment score **S** (or higher) expected to occur **by chance**.

- Smaller E-value, more significant in statistics
- Bigger E-value, less significant
- Over 1 means expect this totally by chance
(not significant at all!)

The expected number of HSPs with the score at least **S** is :

$$E = K * n * m * e^{-\lambda S}$$

K, λ are constant depending on model

n, m are the length of query and sequence

E-values quickly drop off for better alignment bits scores

Very Similar Sequences

Query: HBA_HUMAN Hemoglobin alpha subunit

Sbjct: HBB_HUMAN Hemoglobin beta subunit

Score = 114 bits (285), Expect = 1e-26

Identities = 61/145 (42%), Positives = 86/145 (59%), Gaps = 8/145 (5%)

```
Query   2   LSPADKTNVKAAWGKVGAGHAGEYGAELERMFLSFPTTKTYFPHF-----DLSHGSAQV  55
          L+P +K+ V A WGKV  +  E G EAL R+ + +P T+ +F  F          D   G+ +V
Sbjct   3   LTPEEKSAVTALWGKV--NVDEVGGEALGRLLVVYPWTQRFFESFGDLSTPDVAMGNPKV  60
```

```
Query   56  KGHGKKVADALTNAVAHVDDMPNALSALSDLHAHKLRVDPVNFKLLSHCLLVTLAAHLPA  115
          K HGKKV  A ++ +AH+D++      + LS+LH  KL VDP NF+LL + L+  LA H
Sbjct   61  KAHGKKVLGAFSDGLAHLNCLKGTfATLSELHCDKLHVDPENFRLLGNVLVCVLAHHFGK  120
```

```
Query   116 EFTPAVHASLDKFLASVSTVLTSKY  140
          EFTP V A+  K +A V+  L  KY
Sbjct   121 EFTPPVQAAYQKVVAGVANALAHKY  145
```


Quite Similar Sequences

Query: HBA_HUMAN Hemoglobin alpha subunit

Sbjct: MYG_HUMAN Myoglobin

Score = 51.2 bits (121), Expect = 1e-07,

Identities = 38/146 (26%), Positives = 58/146 (39%), Gaps = 6/146 (4%)

```
Query    2  LSPADKTNVKAAWGKVGAHAGEYGAELERMFLSFPTTKTYFPHF-----DLSHGSAQV    55
          LS  +   V   WGKV A   +G E L R+F   P T   F F       D   S   +
```

```
Sbjct    3  LSDGEWQLVLNVWGKVEADIPGHGQEVLRIRLFKGHHPETLEKFDKFKHLKSEDEMKASEDL    62
```

```
Query   56  KGHGKKVADALTNAAVAHVDDMPNALSALSDLHAHKLRVDPVNFKLLSHCLLVTLAAHLPA    115
          K HG  V  AL   +               +  L+  HA K ++       + +S C++  L +  P
```

```
Sbjct   63  KKHGATVLTALGGILKKKGHHEAEIKPLAQSHATKHKIPVKYLEFISECIIQVLQSKHPG    122
```

```
Query   116  EFTPAVHASLDKFLASVSTVLTSKYR    141
          +F           +++K L       + S Y+
```

```
Sbjct   123  DFGADAQGAMNKALELFRKDMASNYK    148
```

Not similar sequences

Query: HBA_HUMAN Hemoglobin alpha subunit
Sbjct: SPAC869.02c [Schizosaccharomyces pombe]

Score = 33.1 bits (74), Expect = 0.24

Identities = 27/95 (28%), Positives = 50/95 (52%), Gaps = 10/95 (10%)

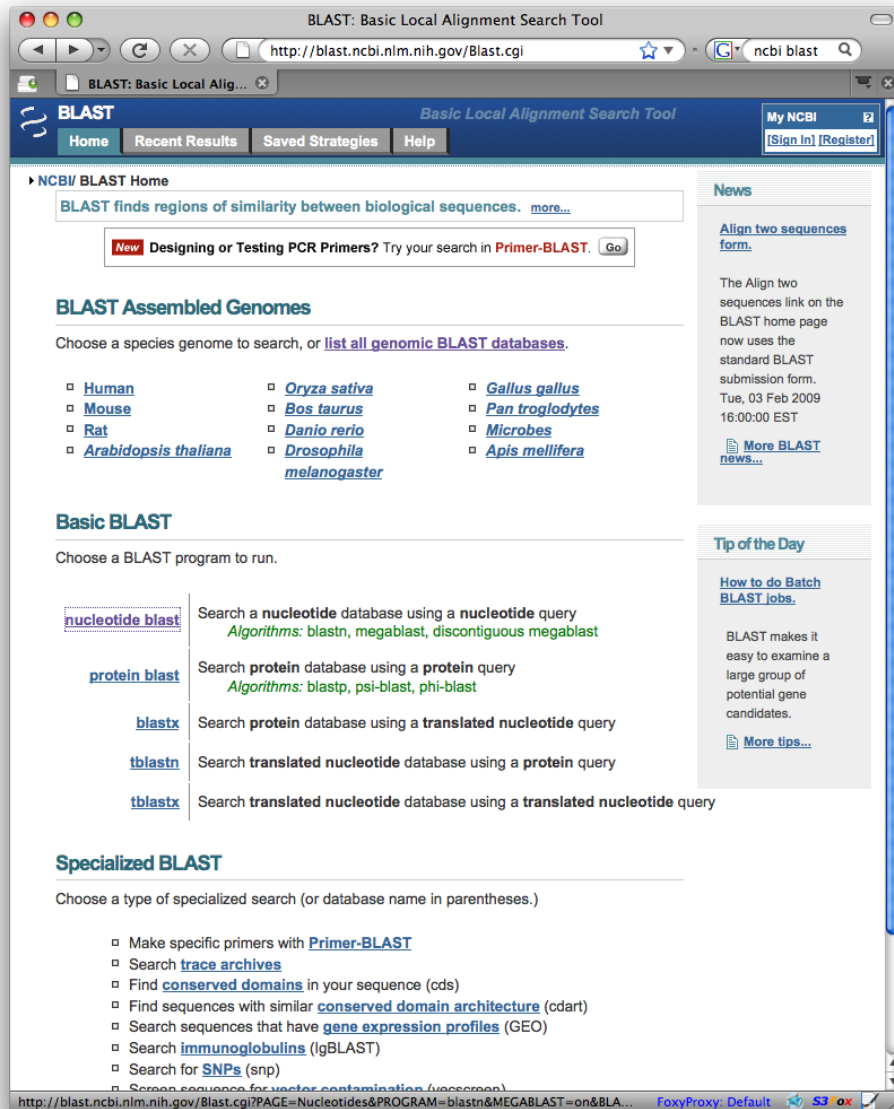
```
Query   30  ERMFLSFPTTKTYFPHFDSLHGSAQVKGHGKKVADALTNAVAHVDDMPNALSALSDLHAH   89
          ++M  ++P          P+F+ +H  +          + +A AL N  ++DD+  +LSA  D
Sbjct   59  QKMLGNYPEV---LPYFNKAHQISL--SQPRILAFALLNYAKNIDDL-TLSAFMDQIVV  112

Query   90  K---LRVDPVNFKLLSHCLLVTLAAHLPAEF-TPA   120
          K   L++   ++ ++ HCLL T+   LP++   TPA
Sbjct  113  KHVGLQIKAEHYPIVGHCLLSTMQELLPSDVATPA   147
```

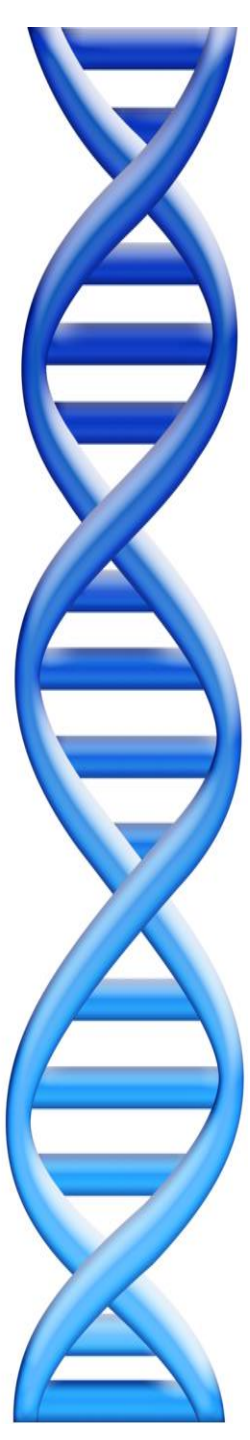
Blast Versions

Program	Database	Query
BLASTN	Nucleotide	Nucleotide
BLASTP	Protein	Protein
BLASTX	Protein	Nucleotide translated into protein
TBLASTN	Nucleotide translated into protein	Protein
TBLASTX	Nucleotide translated into protein	Nucleotide translated into protein

NCBI Blast



- Nucleotide Databases
 - nr:All Genbank
 - refseq: Reference organisms
 - wgs:All reads
- Protein Databases
 - nr:All non-redundant sequences
 - Refseq: Reference proteins



Outline

1. Alignment to other genomes
- 2. Prediction aka “Gene Finding”**
3. Experimental & Functional Assays



Bacterial Gene Finding and Glimmer

(also Archaeal and viral gene finding)

Arthur L. Delcher and Steven Salzberg
Center for Bioinformatics and Computational Biology
Johns Hopkins University

Genetic Code

		Second letter				Third letter
		U	C	A	G	
First letter	U	UUU } Phe UUC } UUA } Leu UUG }	UCU } UCC } Ser UCA } UCG }	UAU } Tyr UAC } UAA Stop UAG Stop	UGU } Cys UGC } UGA Stop UGG Trp	
	C	CUU } CUC } Leu CUA } CUG }	CCU } CCC } Pro CCA } CCG }	CAU } His CAC } CAA } Gln CAG }	CGU } CGC } Arg CGA } CGG }	
	A	AUU } AUC } Ile AUA } AUG Met	ACU } ACC } Thr ACA } ACG }	AAU } Asn AAC } AAA } Lys AAG }	AGU } Ser AGC } AGA } Arg AGG }	
	G	GUU } GUC } Val GUA } GUG }	GCU } GCC } Ala GCA } GCG }	GAU } Asp GAC } GAA } Glu GAG }	GGU } GGC } Gly GGA } GGG }	

Start:
- AUG

Stop:
- UAA
- UAG
- UGA

Step One

- Find open reading frames (ORFs).



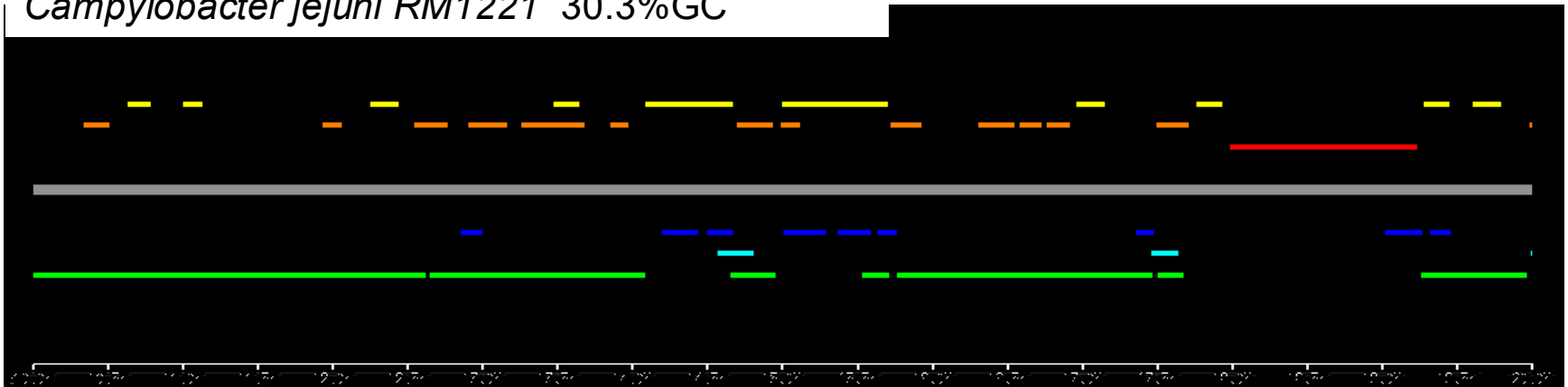
Step One

- Find open reading frames (ORFs).



- But ORFs generally overlap ...

Campylobacter jejuni RM1221 30.3%GC



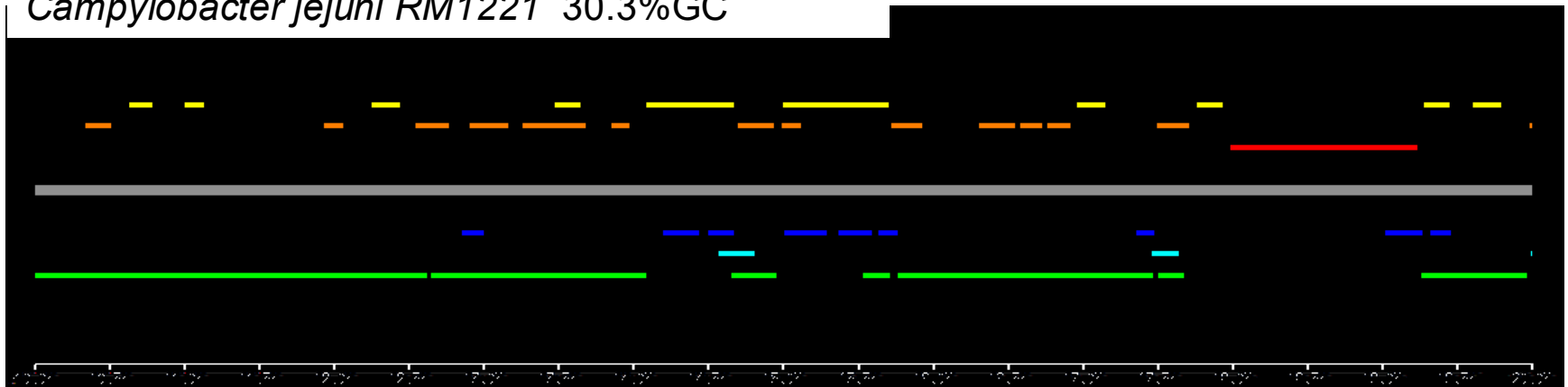
All ORFs longer than 100bp on both strands shown
- color indicates reading frame

Longest ORFs likely to be protein-coding genes

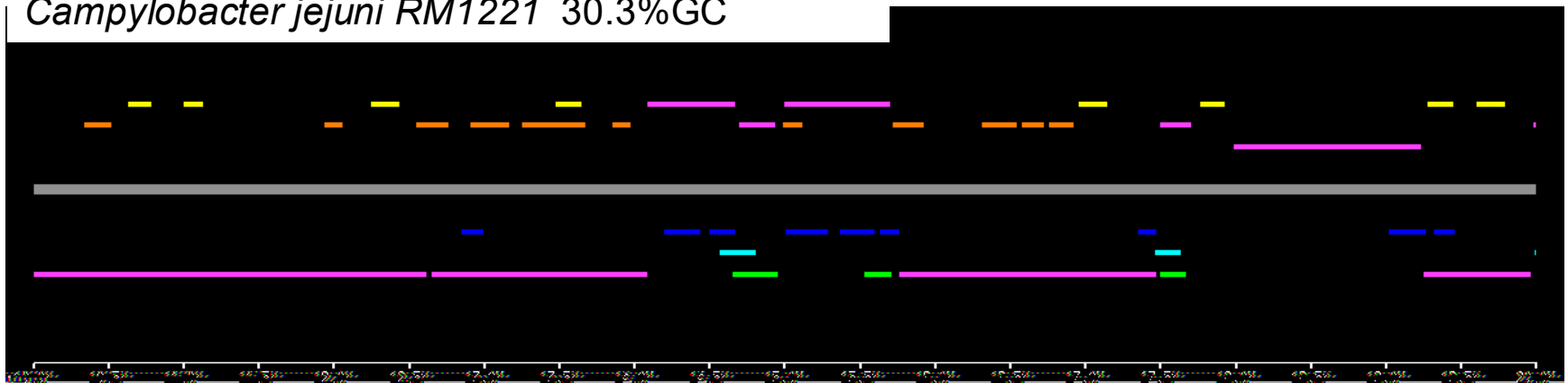
Note the low GC content

All genes are ORFs but not all ORFs are genes

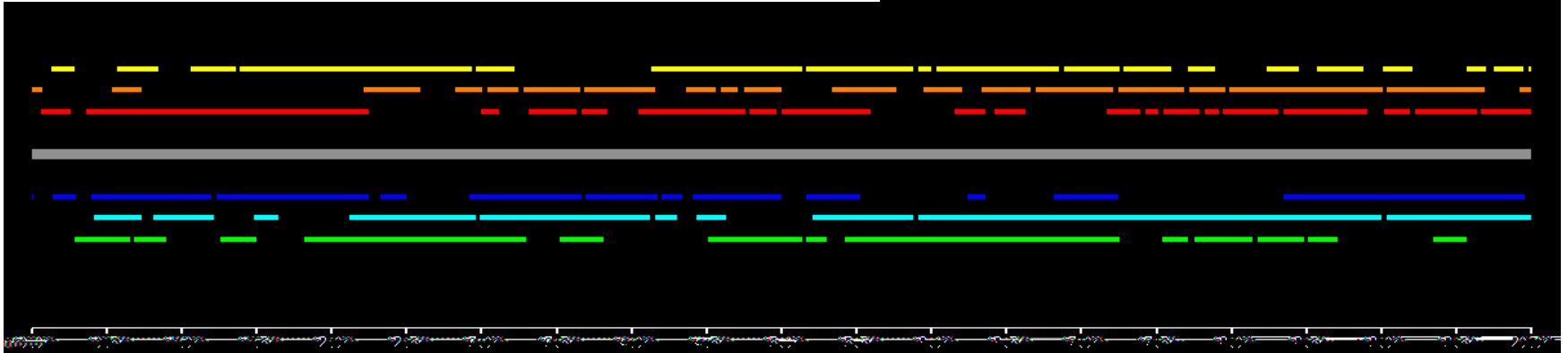
Campylobacter jejuni RM1221 30.3%GC



Campylobacter jejuni RM1221 30.3%GC

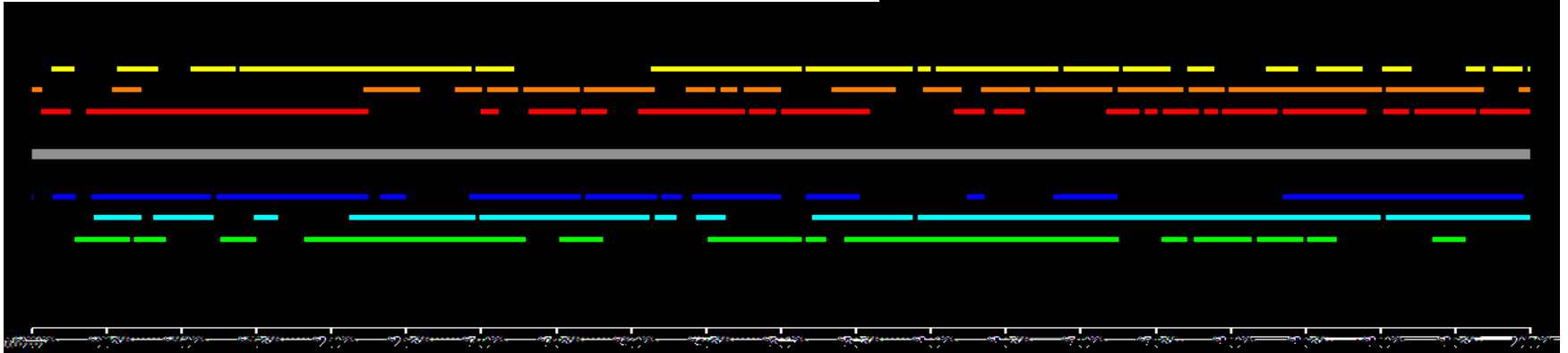


Mycobacterium smegmatis MC2 67.4%GC

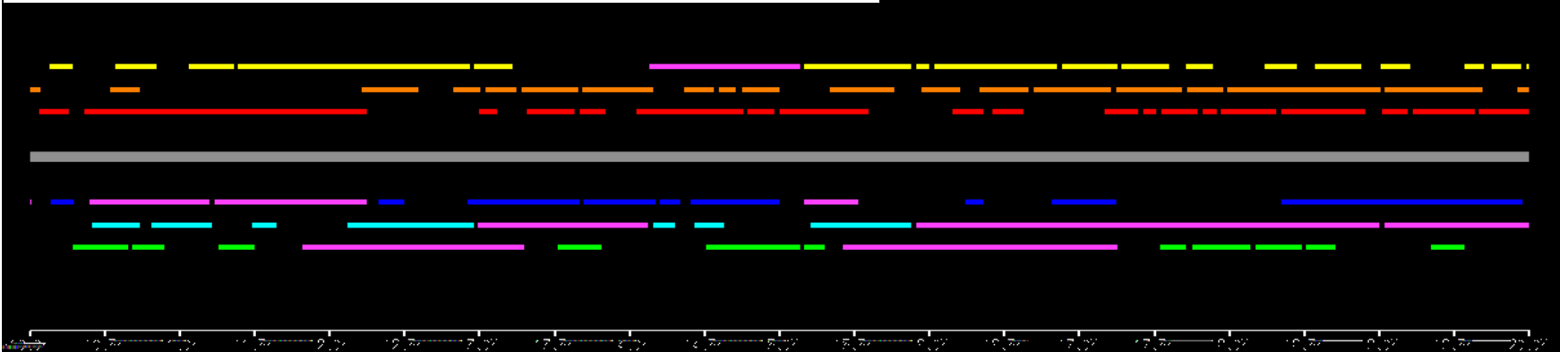


Note what happens in a high-GC genome

Mycobacterium smegmatis MC2 67.4%GC



Mycobacterium smegmatis MC2 67.4%GC



Flipping a Biased Coin

$P(\text{heads}) = 61/64$ (95.4%) $P(\text{tails}) = 3/64$ (4.6%)

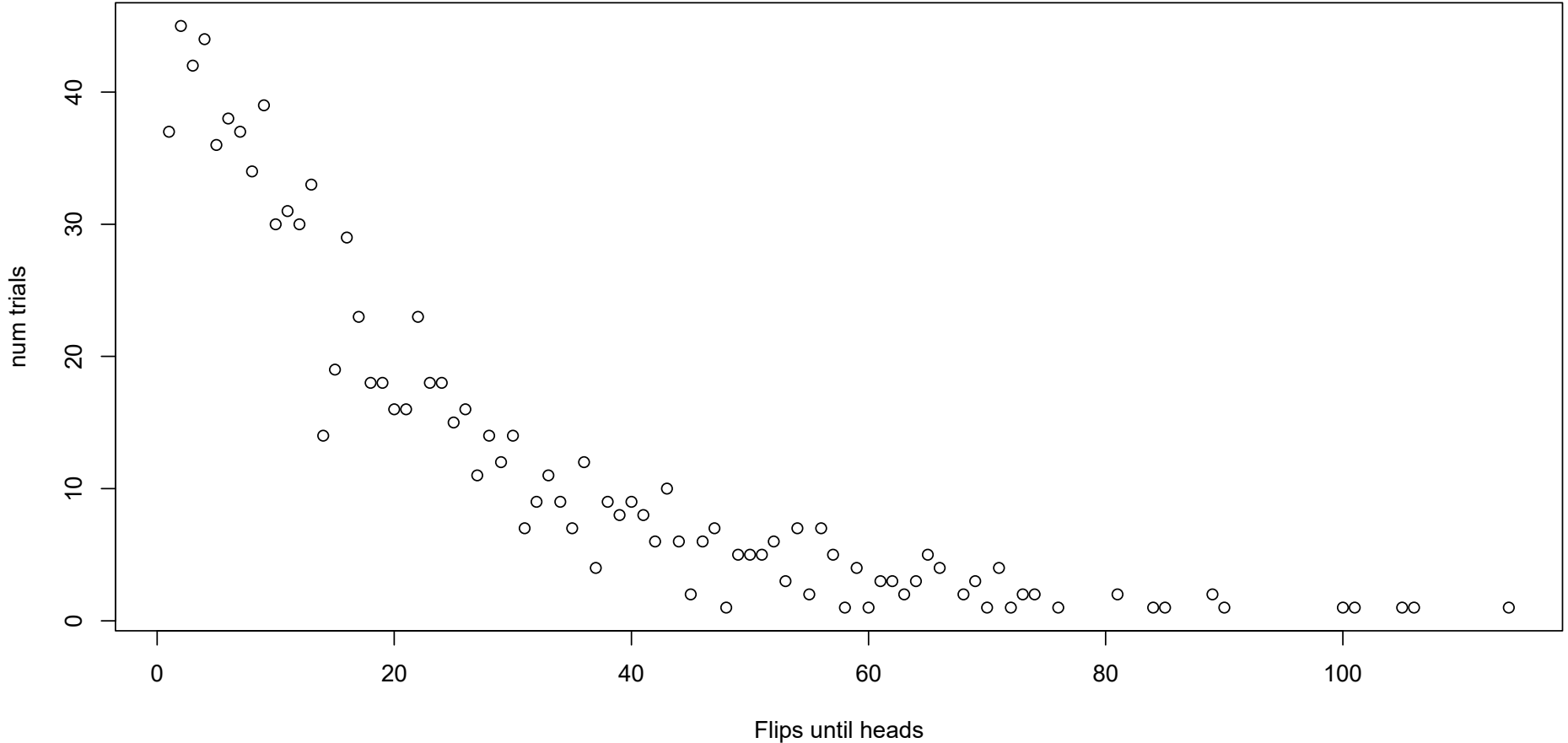
How many flips until my first tail?

[illegible]

Flipping a Biased Coin

$P(\text{heads}) = 61/64$ (95.4%) $P(\text{tails}) = 3/64$ (4.6%)

How many flips until my first tail?

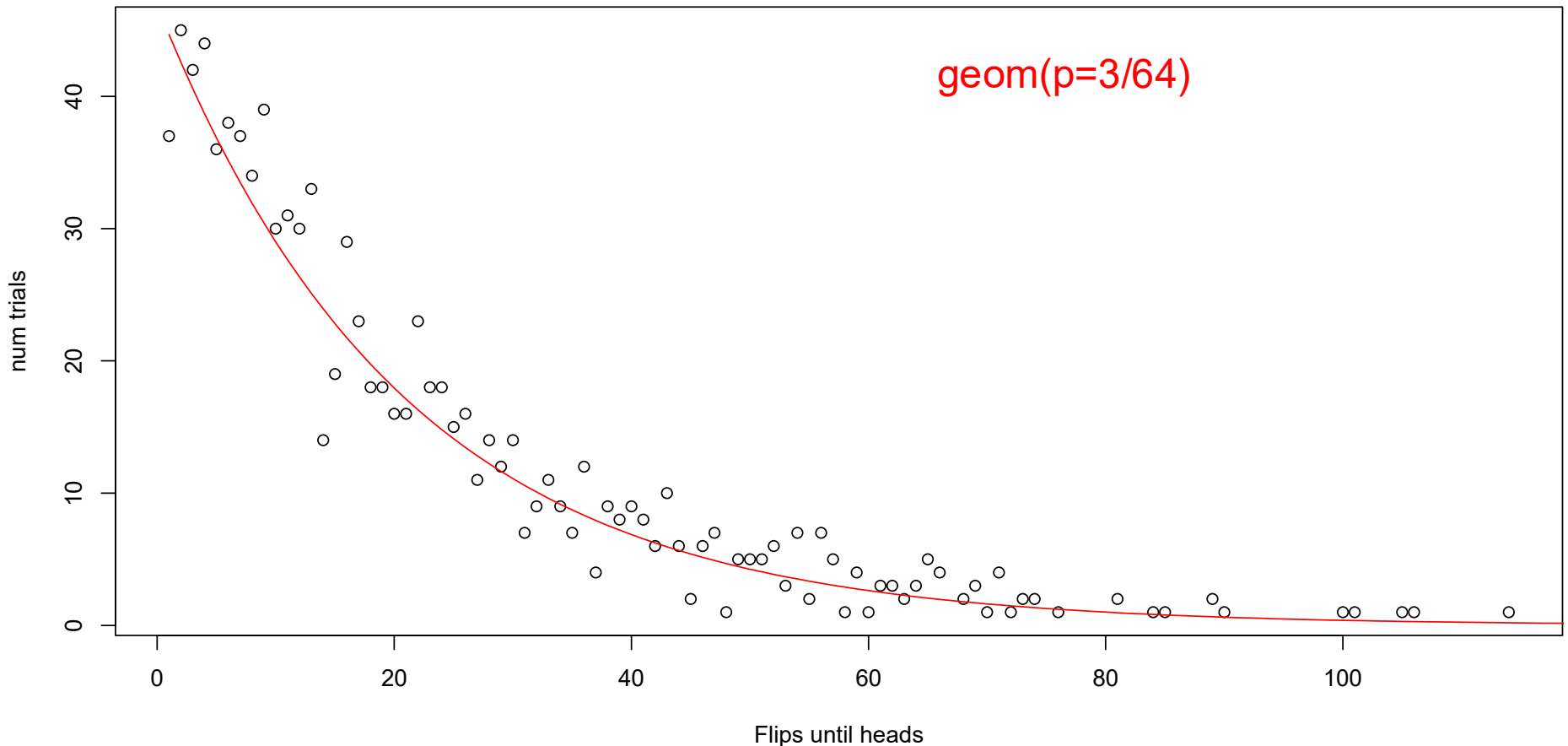


Flipping a Biased Coin

$P(\text{heads}) = 61/64$ (95.4%) $P(\text{tails}) = 3/64$ (4.6%)

How many flips until my first tail?

Geometric Distribution: $P(X=x) = p_{\text{heads}}^{x-1} p_{\text{tails}}$

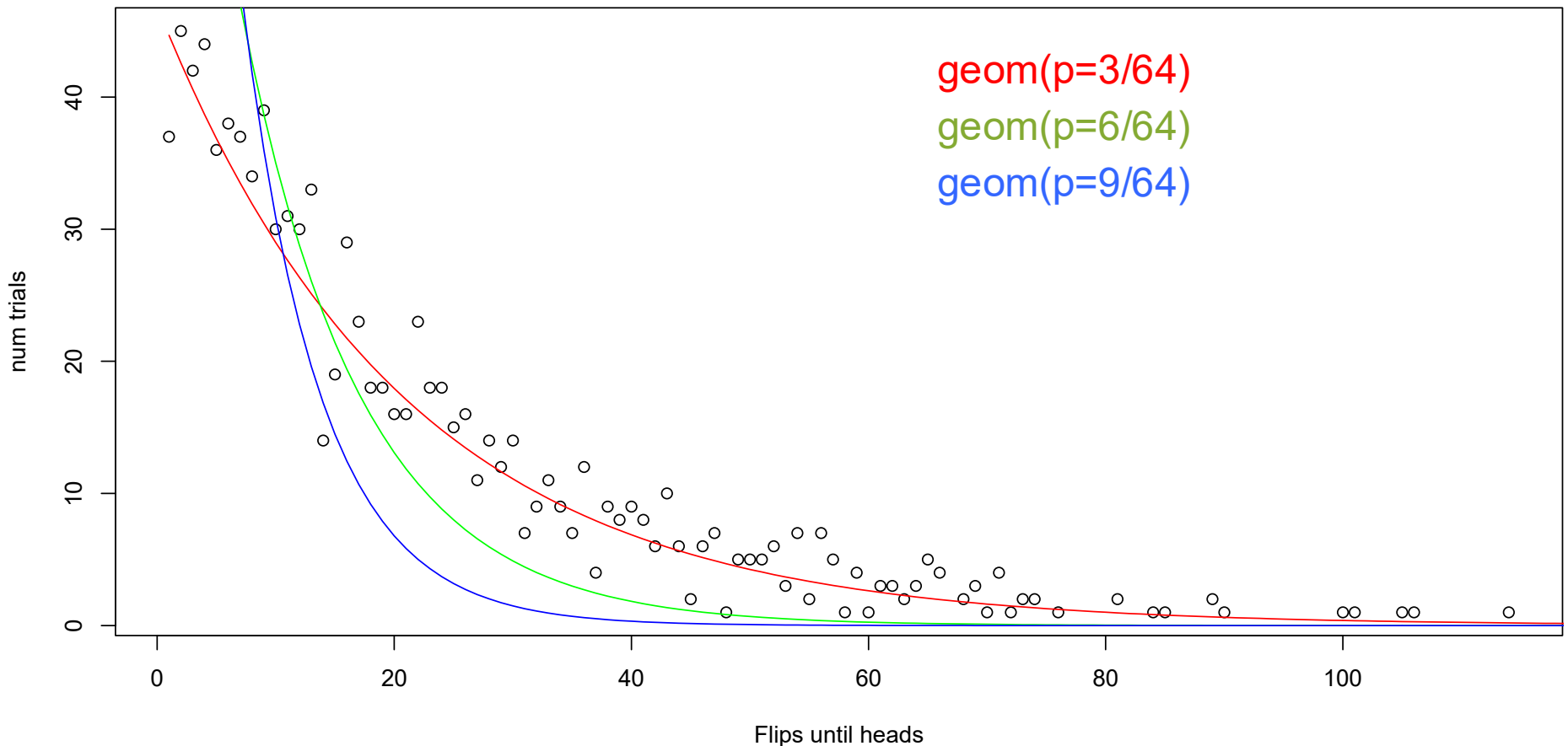


Flipping a Biased Coin

$P(\text{heads}) = 61/64$ (95.4%) $P(\text{tails}) = 3/64$ (4.6%)

How many flips until my first tail?

Geometric Distribution: $P(X=x) = p_{\text{heads}}^{x-1} p_{\text{tails}}$

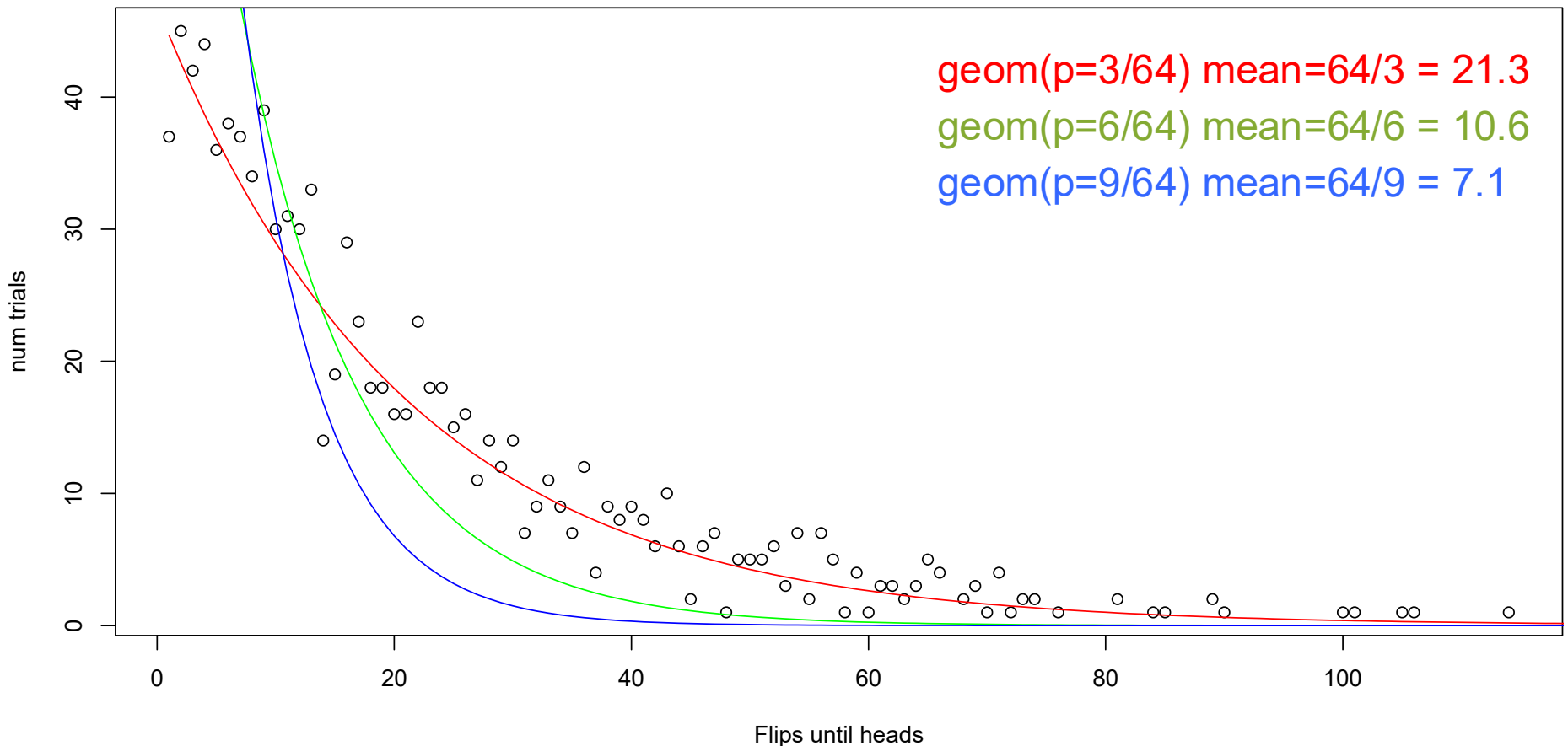


Flipping a Biased Coin

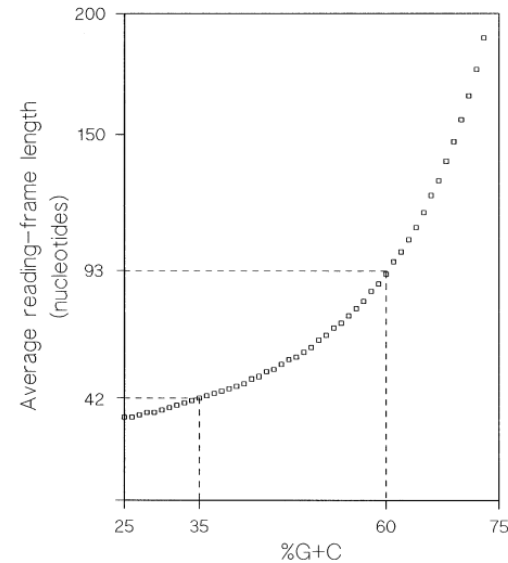
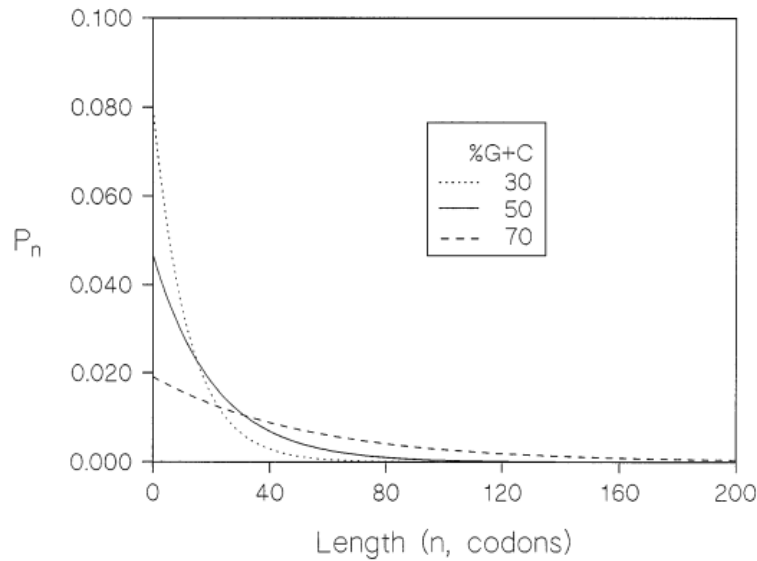
$P(\text{heads}) = 61/64$ (95.4%) $P(\text{tails}) = 3/64$ (4.6%)

How many flips until my first tail?

Geometric Distribution: $P(X=x) = p_{\text{heads}}^{x-1} p_{\text{tails}}$



Stop Codon Frequencies



If the sequence is mostly A+T, then likely to form stop codons by chance!

In High A+T (Low G+C):

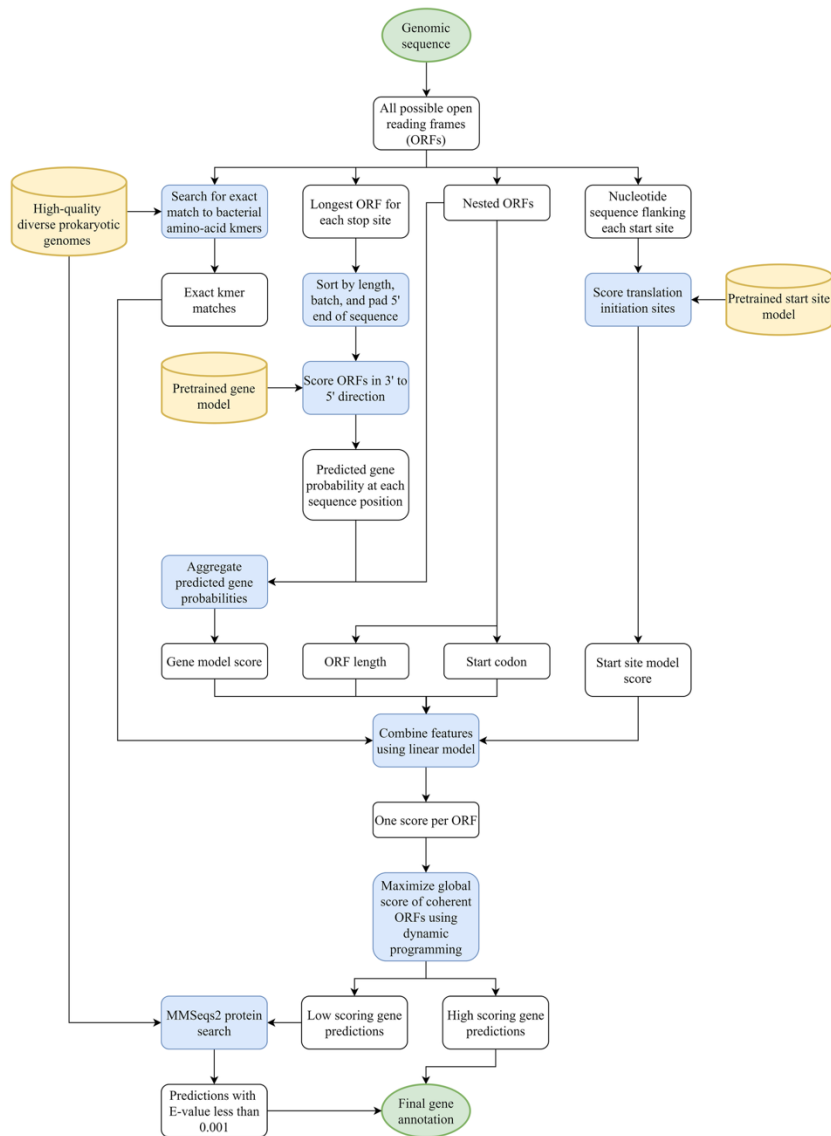
Frequent stop codons; Short Random ORFs; long ORFs likely to be true genes

In High G+C (Low A+T):

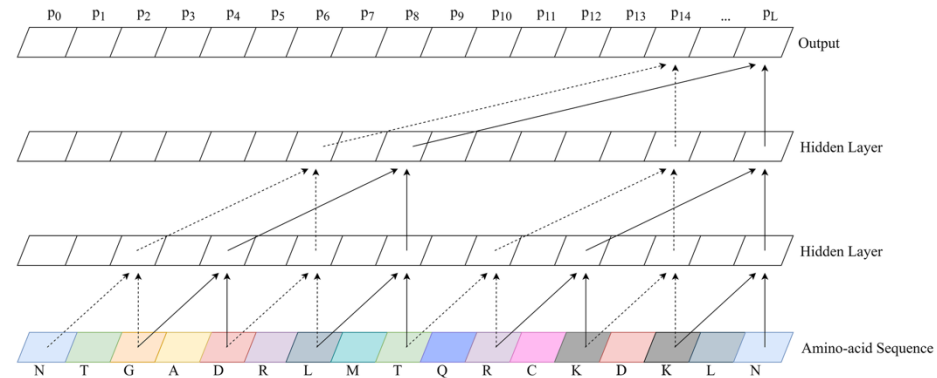
Rare stop codons; Long Random ORFs; harder to identify true genes

A relationship between GC content and coding-sequence length.

Oliver & Marín (1996) J Mol Evol. 43(3):216-23.



Temporal Convolutional Network



Balrog: A universal protein model for prokaryotic gene prediction

Sommer, MJ, Salzberg, SL (2021) PLOS Comp. Bio. doi: 10.1371/journal.pcbi.1008727

Probabilistic Methods

- Create models that have a probability of generating any given sequence.
 - Evaluate gene/non-genome models against a sequence
- Train the models using examples of the types of sequences to generate.
 - Use RNA sequencing, homology, or “obvious” genes
- The “score” of an orf is the probability of the model generating it.
 - Most basic technique is to count how kmers occur in known genes versus intergenic sequences
 - More sophisticated methods consider variable length contexts, “wobble” bases, other statistical clues

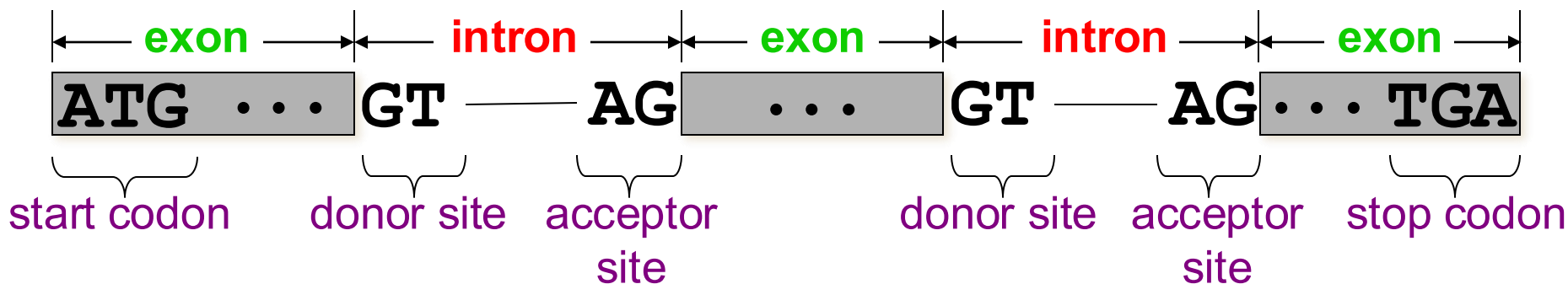
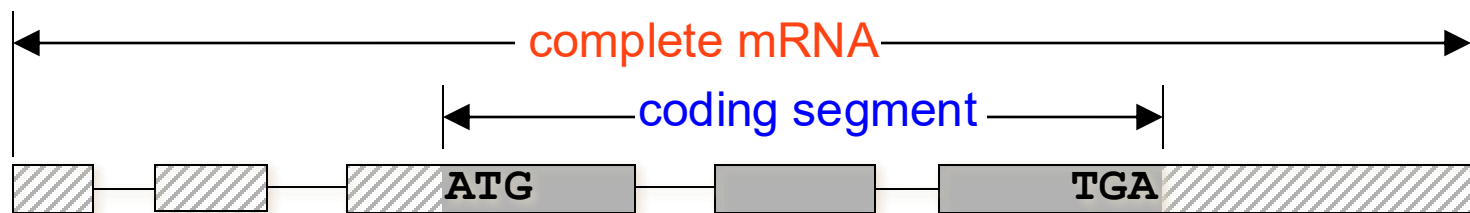


Overview of Eukaryotic Gene Prediction

CBB 231 / COMPSCI 261

W.H. Majoros

Eukaryotic Gene Syntax



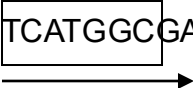
Regions of the gene outside of the CDS are called *UTR*'s (*untranslated regions*), and are mostly ignored by gene finders, though they are important for regulatory functions.

Identifying Signals In DNA

We slide a fixed-length model or “window” along the DNA and evaluate `score(signal)` at each point:

Signal sensor

...ACTGATGCGCGATTAGAGTCATGGCGATGCATCTAGCTAGCTATATCGCGTAGCTAGCTAGCTGATCTACTATCGTAGC...

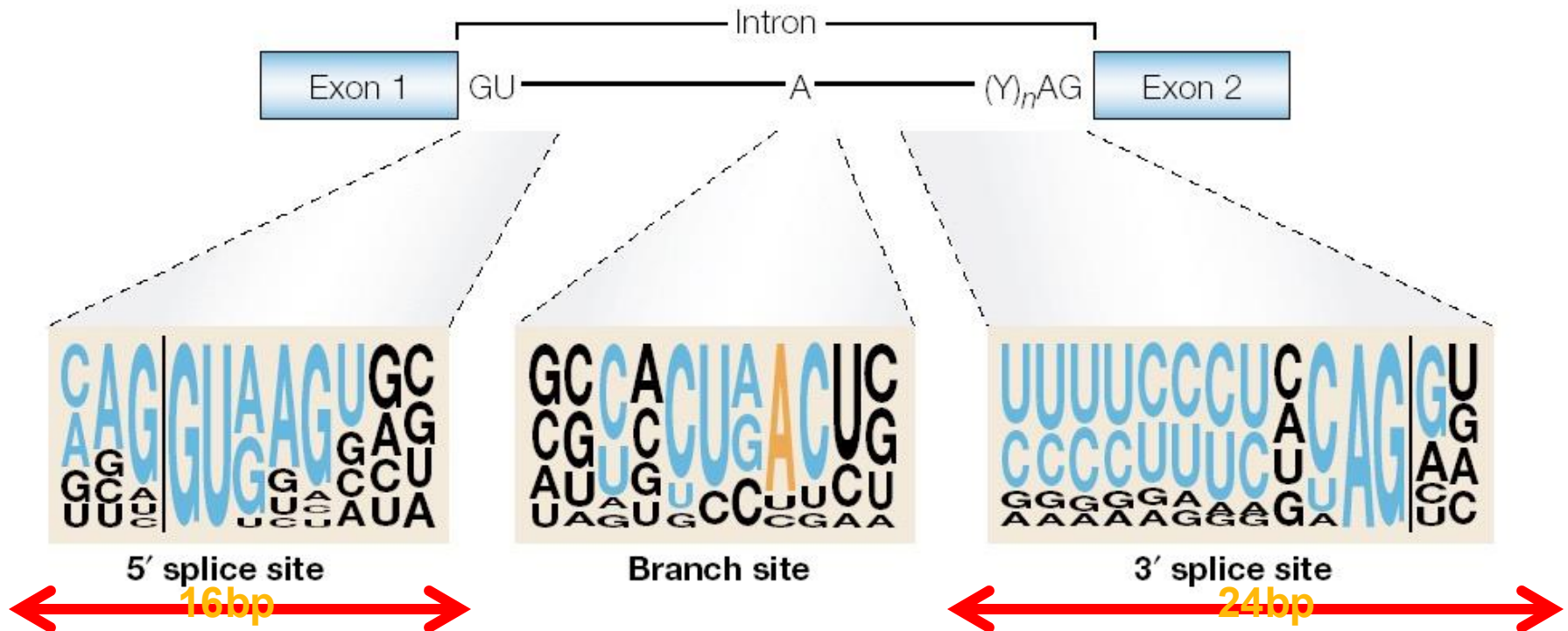


When the `score` is greater than some threshold (determined empirically to result in a desired sensitivity), we remember this position as being the potential site of a signal.

The most common signal sensor is the Position Weight Matrix:

A = 31% T = 28% C = 21% G = 20%	A = 18% T = 32% C = 24% G = 26%	A 100%	T 100%	G 100%	A = 19% T = 20% C = 29% G = 32%	A = 24% T = 18% C = 26% G = 32%
--	--	------------------	------------------	------------------	--	--

Splice site prediction

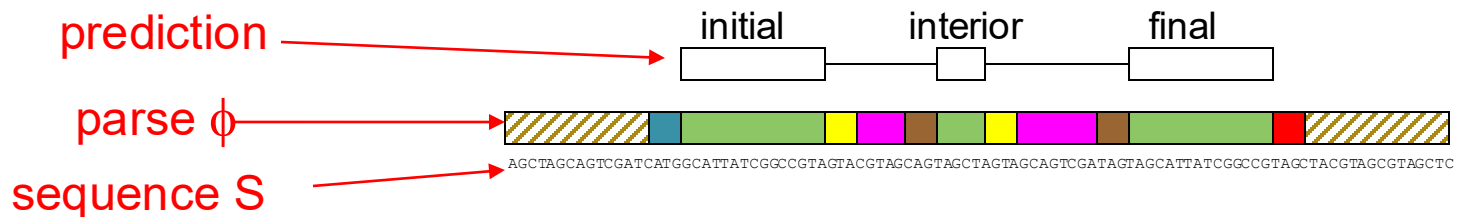


The splice site score is a combination of:

- first or second order inhomogeneous Markov models on windows around the acceptor and donor sites
- Maximal dependence decomposition (MDD) decision trees
- longer Markov models to capture difference between coding and non-coding on opposite sides of site (optional)
- maximal splice site score within 60 bp (optional)

Gene Prediction with a GHMM

Given a sequence S , we would like to determine the parse ϕ of that sequence which segments the DNA into the most likely exon/intron structure:



The parse ϕ consists of the coordinates of the predicted exons, and corresponds to the precise sequence of states during the operation of the GHMM (and their duration, which equals the number of symbols each state emits).

This is the same as in an HMM except that in the HMM each state emits bases with fixed probability, whereas in the GHMM each state emits an entire feature such as an exon or intron.

Genome analysis

Helixer: cross-species gene annotation of large eukaryotic genomes using deep learning

Felix Stiehler¹, Marvin Steinborn¹, Stephan Scholz, Daniela Dey²,
Andreas P. M. Weber¹ and Alisandra K. Denton^{1,*} 

¹Institute of Plant Biochemistry, Faculty of Mathematics and Natural Sciences, Heinrich-Heine-University, Dusseldorf 40225, Germany and ²Institute of Human Genetics, Medical Faculty, RWTH Aachen University, Aachen 52062, Germany

*To whom correspondence should be addressed.

Associate Editor: Inanc Birol

Received on July 31, 2020; revised on November 6, 2020; editorial decision on December 4, 2020; accepted on December 7, 2020

Abstract

Motivation: Current state-of-the-art tools for the *de novo* annotation of genes in eukaryotic genomes have to be specifically fitted for each species and still often produce annotations that can be improved much further. The fundamental algorithmic architecture for these tools has remained largely unchanged for about two decades, limiting learning capabilities. Here, we set out to improve the cross-species annotation of genes from DNA sequence alone with the help of deep learning. The goal is to eliminate the dependency on a closely related gene model while also improving the predictive quality in general with a fundamentally new architecture.

Results: We present Helixer, a framework for the development and usage of a cross-species deep learning model that improves significantly on performance and generalizability when compared to more traditional methods. We evaluate our approach by building a single vertebrate model for the base-wise annotation of 186 animal genomes and a separate land plant model for 51 plant genomes. Our predictions are shown to be much less sensitive to the length of the genome than those of a current state-of-the-art tool. We also present two novel post-processing techniques that each worked to further strengthen our annotations and show in-depth results of an RNA-Seq based comparison of our predictions. Our method does not yet produce comprehensive gene models but rather outputs base pair wise probabilities.

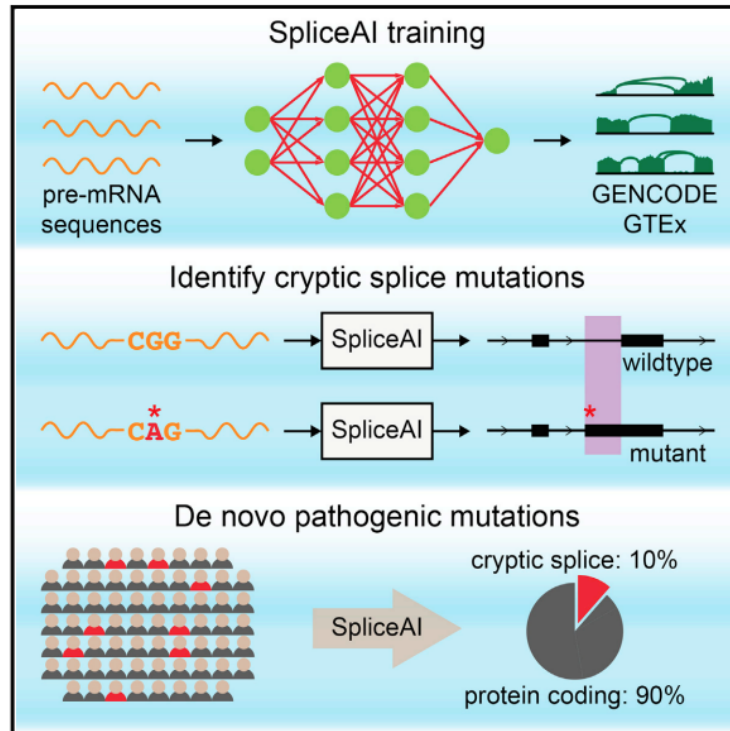
Availability and implementation: The source code of this work is available at <https://github.com/weberlab-hhu/Helixer> under the GNU General Public License v3.0. The trained models are available at <https://doi.org/10.5281/zenodo.3974409>

Contact: alisandra.denton@hhu.de

Supplementary information: [Supplementary data](#) are available at *Bioinformatics* online.

Predicting Splicing from Primary Sequence with Deep Learning

Graphical Abstract



Authors

Kishore Jaganathan,
Sofia Kyriazopoulou Panagiotopoulou,
Jeremy F. McRae, ..., Serafim Batzoglou,
Stephan J. Sanders, Kyle Kai-How Farh

Correspondence

kfarh@illumina.com

In Brief

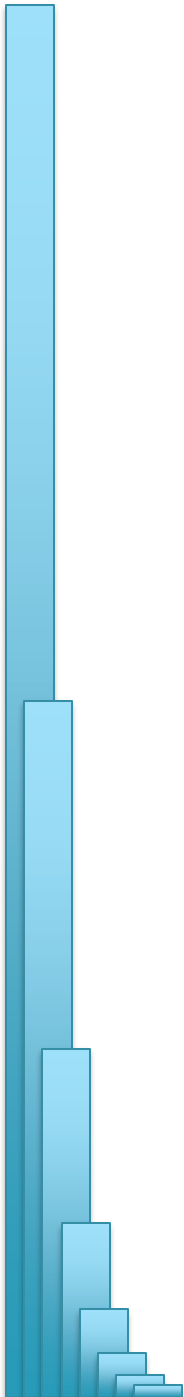
A deep neural network precisely models mRNA splicing from a genomic sequence and accurately predicts noncoding cryptic splice mutations in patients with rare genetic diseases.

Highlights

- SpliceAI, a 32-layer deep neural network, predicts splicing from a pre-mRNA sequence
- 75% of predicted cryptic splice variants validate on RNA-seq
- Cryptic splicing may yield ~10% of pathogenic variants in neurodevelopmental disorders
- Cryptic splice variants frequently give rise to alternative splicing

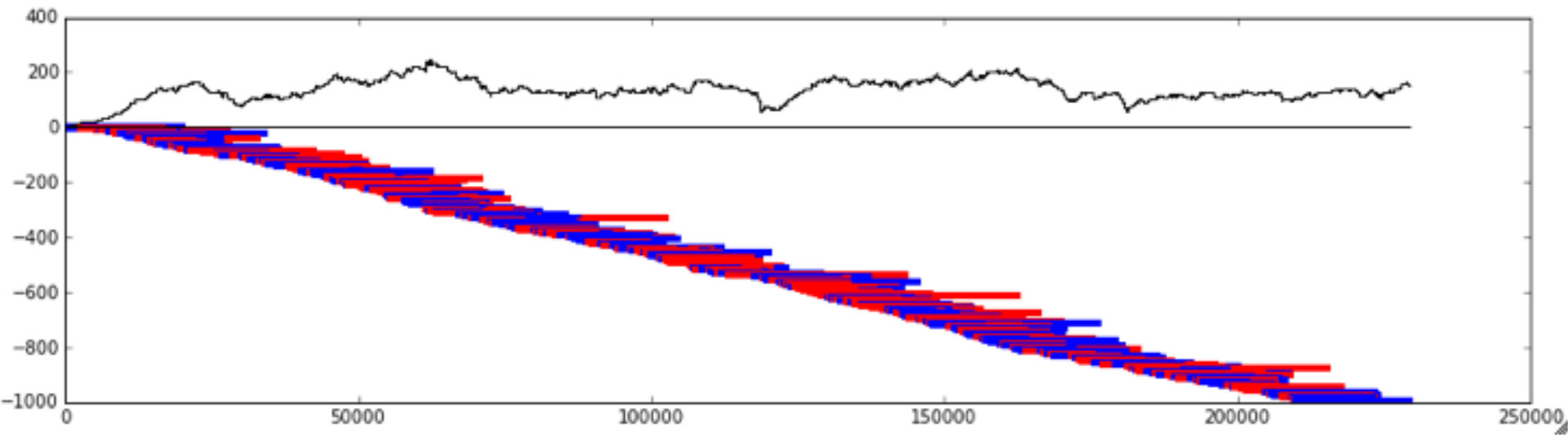
Gene Finding Overview

- Prokaryotic gene finding distinguishes real genes and random ORFs
 - Prokaryotic genes have simple structure and are largely homogenous, making it relatively easy to recognize their sequence composition
- Eukaryotic gene finding identifies the genome-wide most probable gene models (set of exons)
 - “Probabilistic Graphical Model” to enforce overall gene structure, separate models to score splicing/transcription signals
 - Accuracy depends to a large extent on the quality of the training data



Plane Sweep Algorithm

Coverage across the genome



```
$ head -3 ~/readid.start.stop.txt
```

```
1 0 19814
```

```
2 799 19947
```

```
3 1844 13454
```

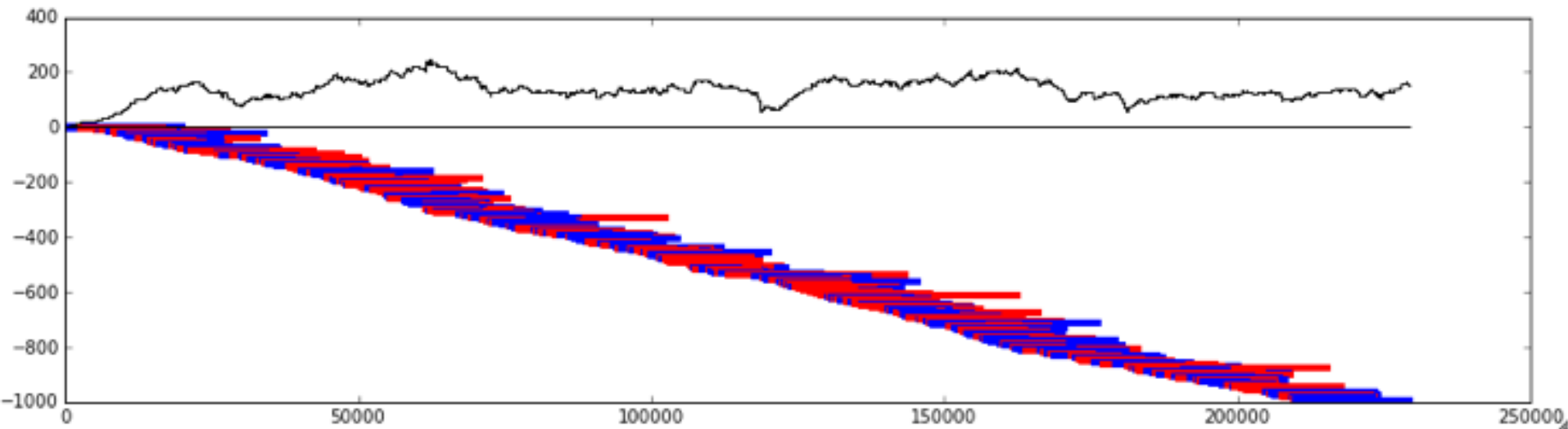
```
$ tail -3 ~/readid.start.stop.txt
```

```
1871 973590 965902
```

```
1872 966703 973521
```

```
1873 973632 966946
```

Coverage across the genome



```
print "Plotting layout"
```

```
## draw the layout of reads
```

```
for i in xrange(min(MAX_READS_LAYOUT, len(reads))):
```

```
    r = reads[i]
```

```
    readid = r[0]
```

```
    start  = r[1]
```

```
    end    = r[2]
```

```
    rc     = r[3]
```

```
    color  = "blue"
```

```
    if (rc == 1):
```

```
        color = "red"
```

```
plt.plot ([start,end], [-2*i, -2*i], lw=4, color=color)
```

←
r[1] is start pos
r[2] is end pos

Brute Force Coverage Profile

```
print "Brute force computing coverage over %d bp" % (totallen)

starttime = time.time()
brutecov = [0] * totallen

for r in reads:
    # print " -- [%d, %d]" % (r[1], r[2])

    for i in xrange(r[1], r[2]):
        brutecov[i] += 1

brutetime = (time.time() - starttime) * 1000.0

print "  Brute force complete in %0.02f ms" % (brutetime)
print brutecov[0:10]
```

Add 1 to coverage
vector at every
position the read
covers

```
Brute force computing coverage over 973898 bp
  Brute force complete in 4435.00 ms
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

Notice that it took 4435 ms for this to complete

Delta Encoding aka run length encoding

```
deltacov = []
curcov = -1
for i in xrange(0, len(brutecov)):
    if brutecov[i] != curcov:
        curcov = brutecov[i]
        delta = (i, curcov)
        deltacov.append(delta)
```

```
## Finish up with the last position
deltacov.append((totallen, 0))
```

Only record those positions when the coverage changes

Delta encoding coverage plot

Delta encoding required only 3697 steps, saving 99.62% of the space in 151.32 ms

```
0: [0,1]
1: [799,2]
2: [1844,3]
...
3694: [973770,2]
3695: [973779,1]
3696: [973898,0]
```

Plot Coverage and Read Positions

```
## expand the coverage profile by this amount so that it is easier to see
YSCALE = 5
```

```
## draw the layout of reads
```

```
for i in xrange(min(MAX_READS_LAYOUT, len(reads))):
    r = reads[i]
    readid = r[0]
    start = r[1]
    end = r[2]
    rc = r[3]
    color = "blue"

    if (rc == 1):
        color = "red"
```

```
plt.plot([start,end], [-2*i, -2*i], lw=4, color=color)
```

```
## draw the base of the coverage plot
```

```
plt.plot([0, totalen], [0,0], color="black")
```

```
## draw the coverage plot
```

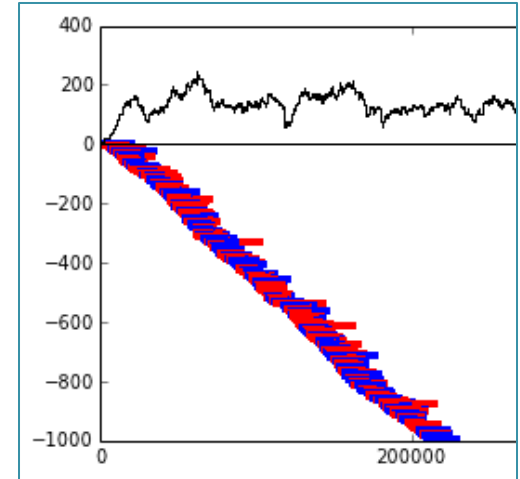
```
for i in xrange(len(deltacov)-1):
    x1 = deltacov[i][0]
    x2 = deltacov[i+1][0]
    y1 = YSCALE*deltacov[i][1]
    y2 = YSCALE*deltacov[i+1][1]
```

```
## draw the horizontal line
```

```
plt.plot([x1, x2], [y1, y1], color="black")
```

```
## and now the right vertical to the new coverage level
```

```
plt.plot([x2, x2], [y1, y2], color="black")
```



Plot Each Read

Plot Each
Coverage Step

Plot Coverage and Read Positions

```
## expand the coverage profile by this amount so that it is easier to see  
YSCALE = 5
```

```
## draw the layout of reads
```

```
for i in xrange(min(MAX_READS_LAYOUT, len(reads))):  
    r = reads[i]  
    readid = r[0]  
    start = r[1]  
    end = r[2]  
    rc = r[3]  
    color = "blue"  
  
    if (rc == 1):  
        color = "red"
```

```
plt.plot([start,end], [-2*i, -2*i], lw=4, color=color)
```

```
## draw the base of the coverage plot
```

```
plt.plot([0, totalen], [0,0], color="black")
```

```
## draw the coverage plot
```

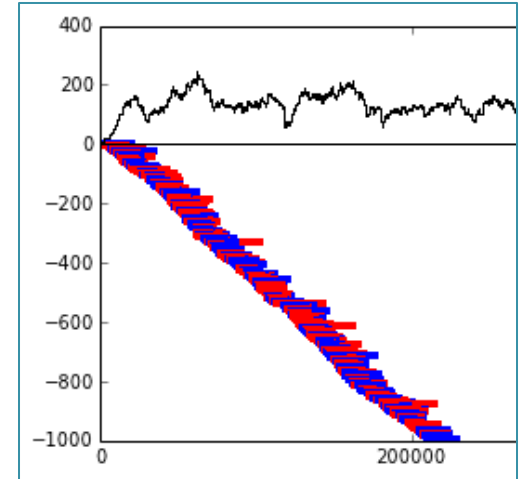
```
for i in xrange(len(deltacov)):  
    x1 = deltacov[i][0]  
    x2 = deltacov[i+1][0]  
    y1 = YSCALE*deltacov[i][1]  
    y2 = YSCALE*deltacov[i+1][1]
```

```
## draw the horizontal line
```

```
plt.plot([x1, x2], [y1, y1], color="black")
```

```
## and now the right vertical to the new coverage level
```

```
plt.plot([x2, x2], [y1, y2], color="black")
```



Plot Each Read

Brute Force works, but is pretty slow.

How can we make it go faster?

Plot Each
Coverage Step

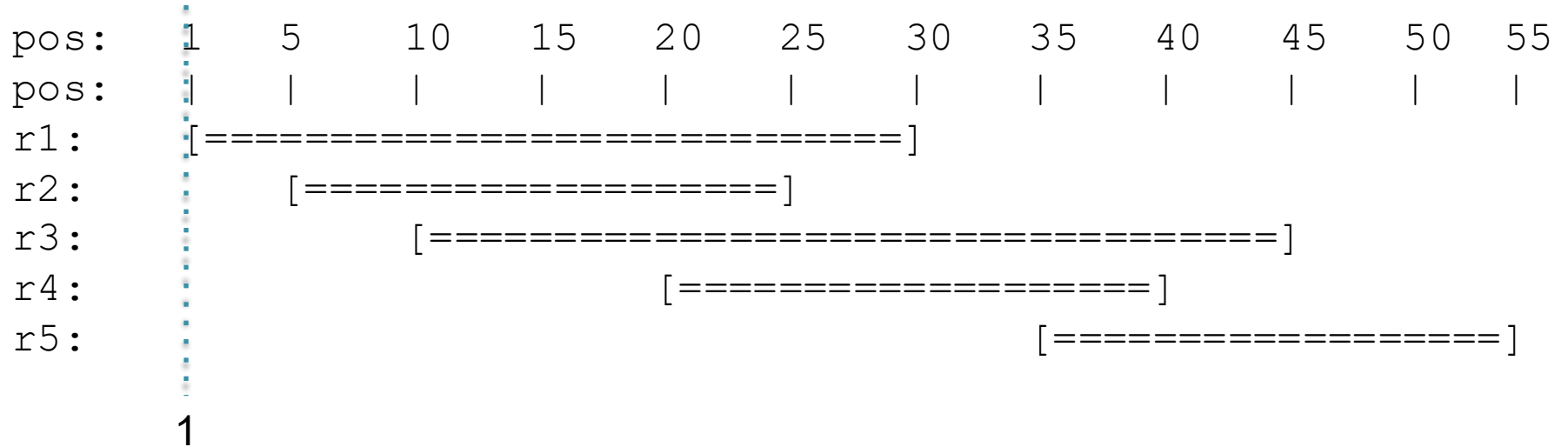
Plane Sweep

```
pos:   1    5    10   15   20   25   30   35   40   45   50   55
pos:   |    |    |    |    |    |    |    |    |    |
r1:    [=====]
r2:          [=====]
r3:                [=====]
r4:                      [=====]
r5:                                [=====]
```

The basic algorithm works like this:

- Assume layout is in sorted order by start position (or explicitly sort by start position)
- use a “list” to track how many reads currently intersect the plane keyed by end coord
 - the number of elements in the list corresponds to the current depth
- walking from start position to start position
 - check to see if we past any read ends
 - coverage goes down by one when a read ends
 - coverage goes up by one when new read is encountered

Plane Sweep

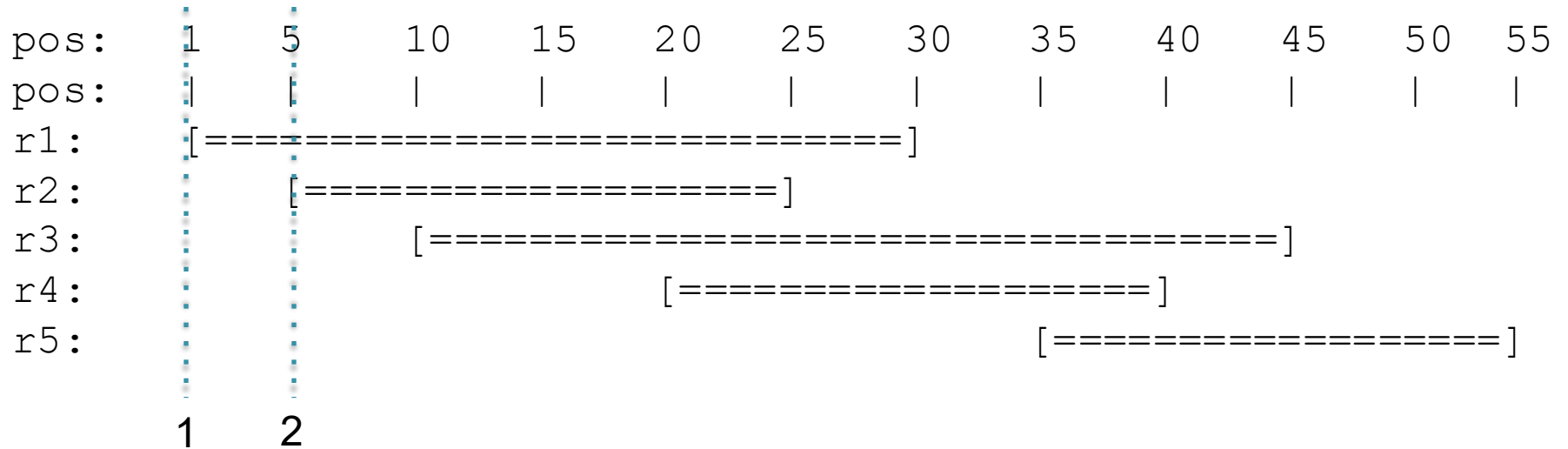


arrive at r1 [1,30]:

active set is empty; add to active set: 30

output (1,1)

Plane Sweep



arrive at r1 [1,30]:

active set is empty; add to active set: 30

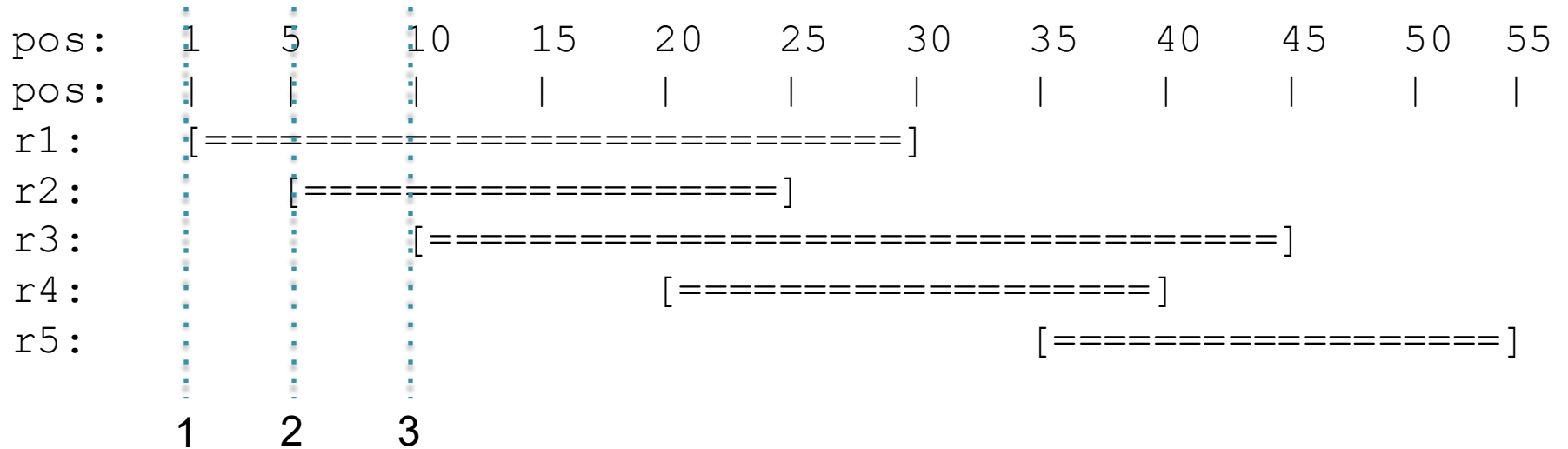
output (1,1)

arrive at r2 [5,25]:

5 < 30: add to active set: 25, 30 <- notice insert out of order

output (5, 2)

Plane Sweep



arrive at r1 [1,30]:

active set is empty; add to active set: 30

output (1,1)

arrive at r2 [5,25]:

5 < 30: add to active set: 25, 30 <- notice insert out of order

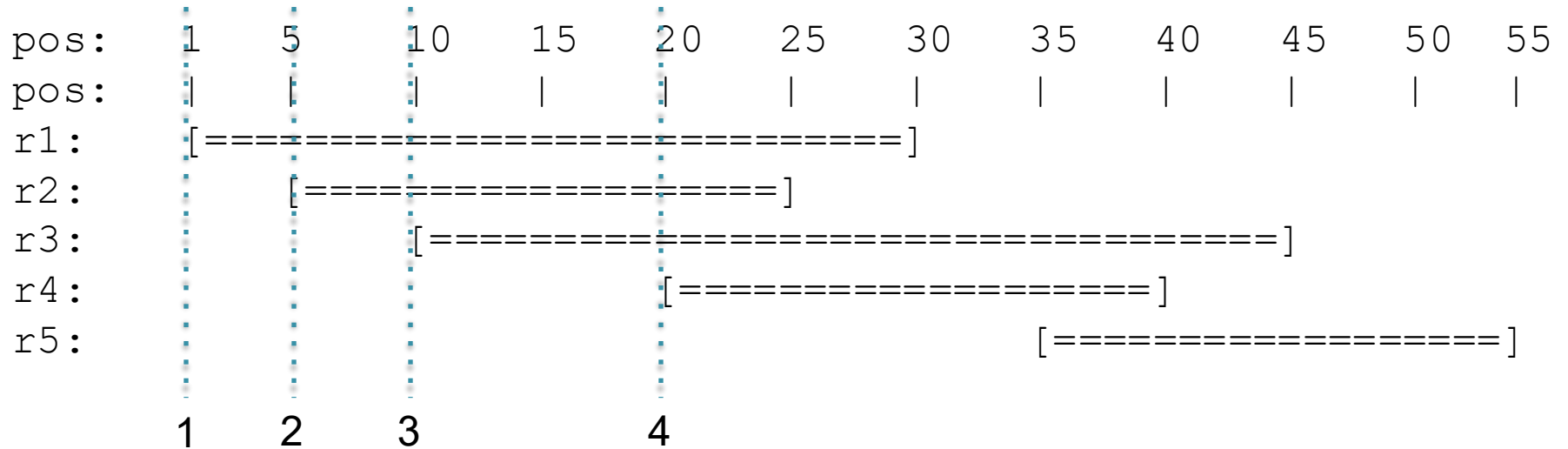
output (5, 2)

arrive at r3 [10,45]:

10 < 25; add to active set: 25, 30, 45

output (10, 3)

Plane Sweep



arrive at r3 [10,45]:

10 < 25; add to active set: 25, 30, 45

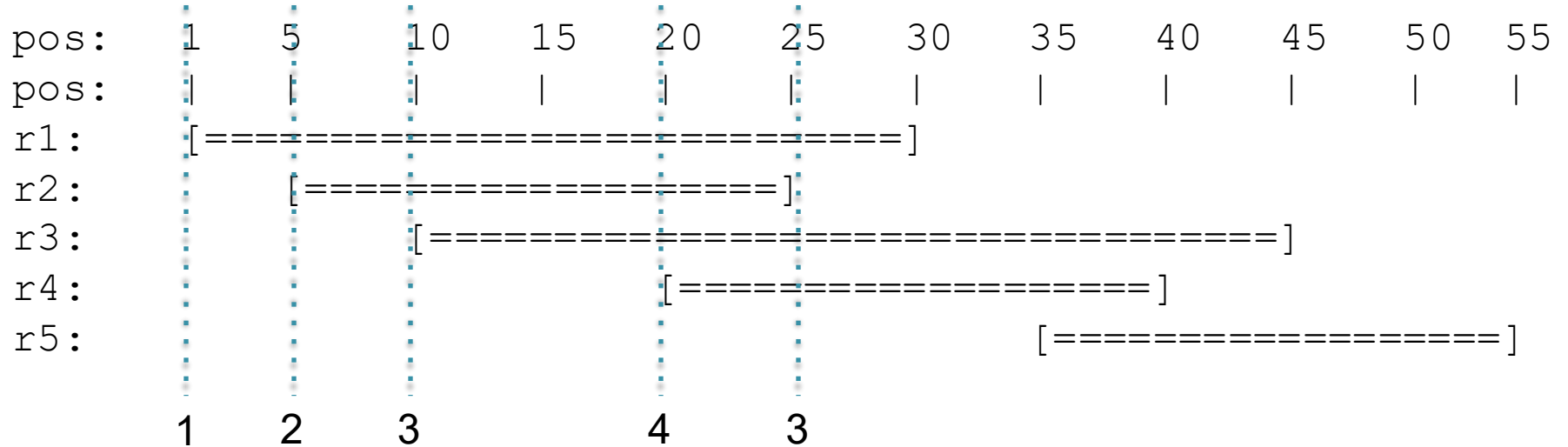
output (10, 3)

arrive at r4 [20,40]:

20 < 25; add to active set: 25, 30, 40, 45 <- out of order again

output (20, 4)

Plane Sweep

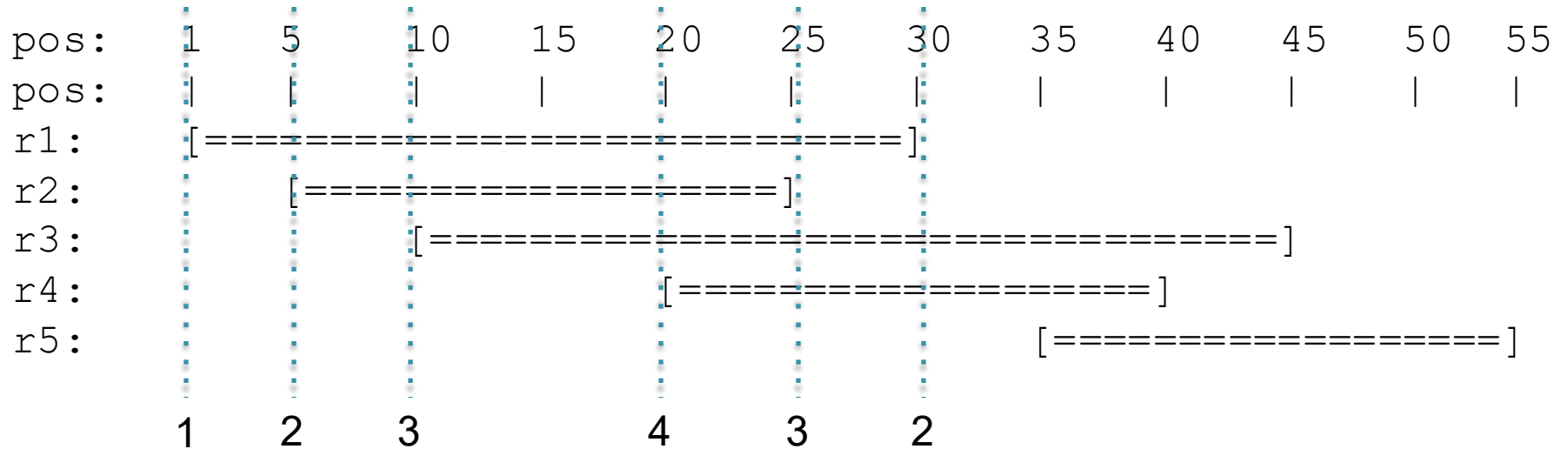


arrive at r5[35,55]:

35 > 25: step down at 25; active set: 30, 40, 45

output (25, 3)

Plane Sweep



arrive at r5[35,55]:

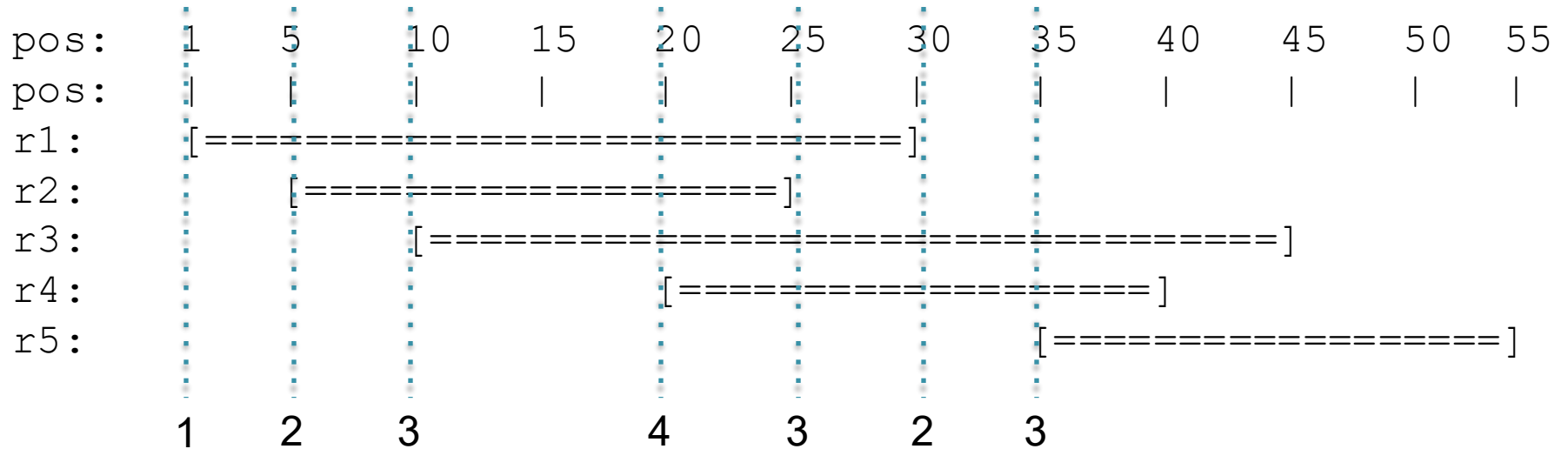
35 > 25: step down at 25; active set: 30, 40, 45

output (25, 3)

35 > 30: step down at 30; active set: 40, 45

output (30, 2)

Plane Sweep



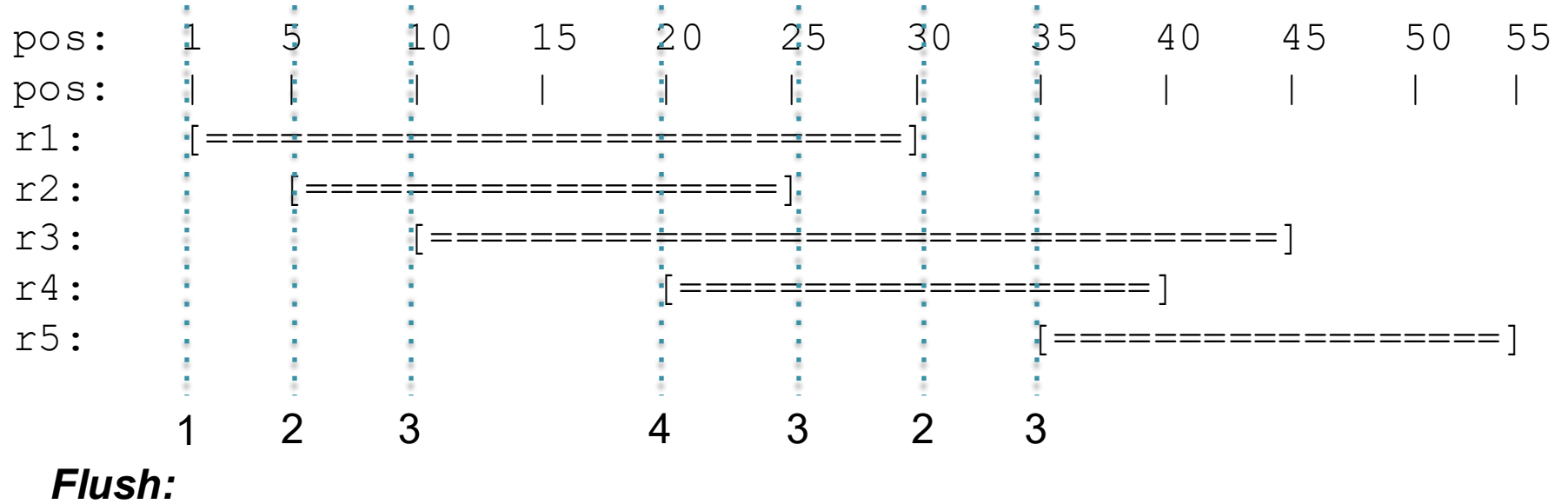
arrive at r5[35,55]:

35 > 25: step down at 25; active set: 30, 40, 45
output (25, 3)

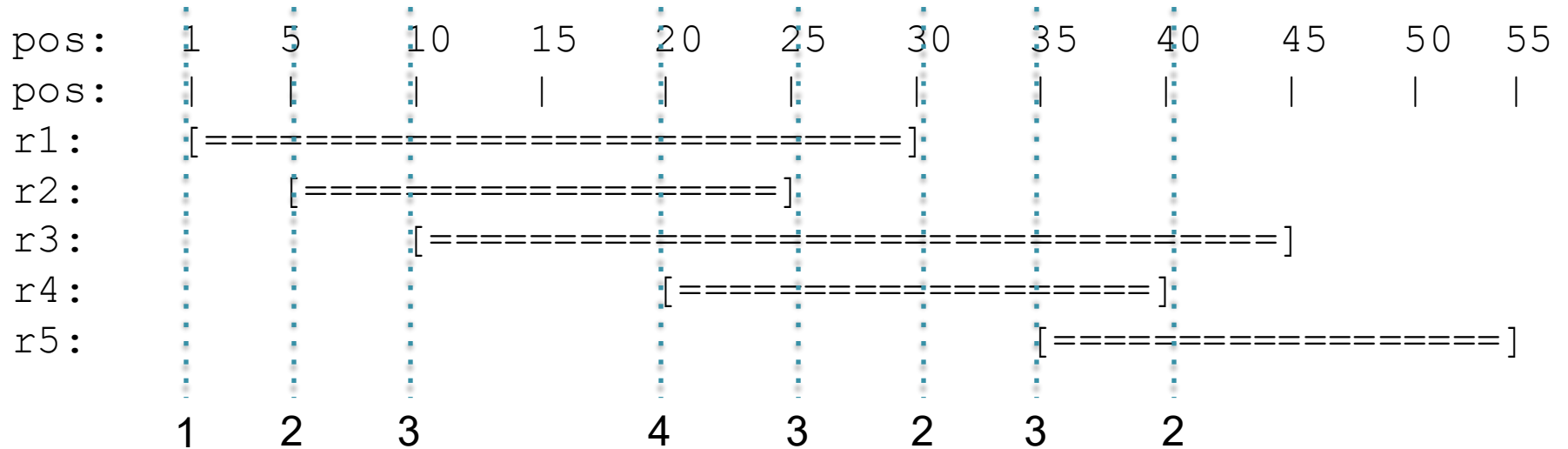
35 > 30: step down at 30; active set: 40, 45
output (30, 2)

35 < 40: add to active set: 40, 45, 55
output (35, 3)

Plane Sweep



Plane Sweep

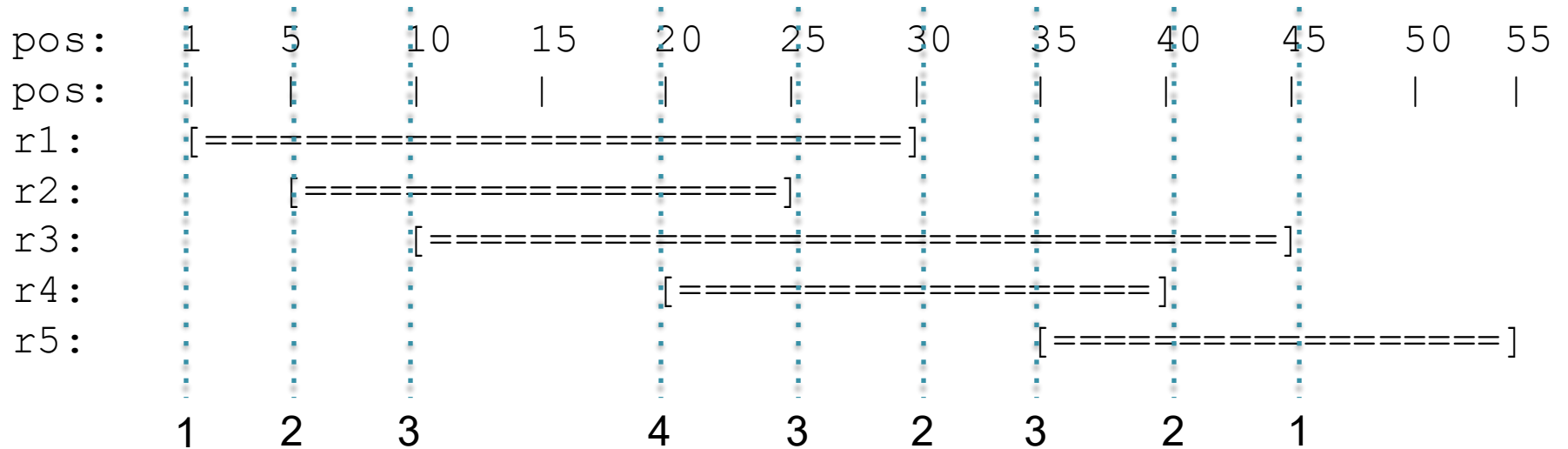


Flush:

step down at 40; active set: 45, 55

output (40, 2)

Plane Sweep

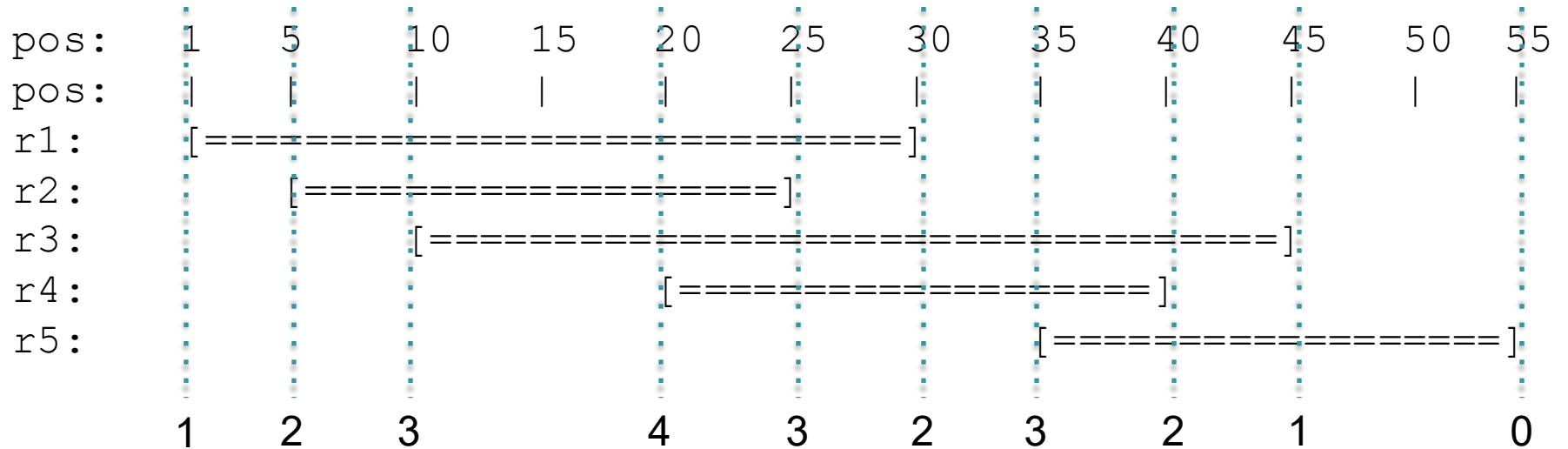


Flush:

step down at 40; active set: 45, 55
output (40, 2)

step down at 45: active set: 55
output (45, 1)

Plane Sweep



Flush:

step down at 40; active set: 45, 55
output (40, 2)

step down at 45: active set: 55
output (45, 1)

step down at 55: active set: {}
output (55, 0)

Plane Sweep

```
## record the delta encoded depth using a plane sweep
deltacovplane = []

## use a list to record the end positions of the elements currently in plane
planelist = []

## BEGIN SWEEP
for r in reads:
    startpos = r[1]
    endpos   = r[2]

    ## clear out any positions from the plane that we have already moved past
    while (len(planelist) > 0):

        if (planelist[0] <= startpos):
            ## the coverage steps down, extract it from the front of the list
            oldend = planelist.pop(0)
            deltacovplane.append((oldend, len(planelist)))
        else:
            break

    ## Now insert the current endpos into the correct position into the list
    insertpos = -1
    for i in xrange(len(planelist)):
        if (endpos < planelist[i]):
            insertpos = i
            break

    if (insertpos > 0):
        planelist.insert(insertpos, endpos)
    else:
        planelist.append(endpos)

    ## Finally record that the coverage has increased
    deltacovplane.append((startpos, len(planelist)))

## Flush any remaining end positions
while (len(planelist) > 0):
    oldend = planelist.pop(0)
    deltacovplane.append((oldend, len(planelist)))
```

Keep track of end positions of reads that have been seen so far

Check to see if any reads have ended before the start of this one

Add the end of the current read to the sweep plane in sorted order

Why sorted?

Beginning list-based plane sweep over 1873 reads
Plane sweep found 3746 steps, saving 99.62% of the space in 48.90 ms (90.69 speedup)!

Plane Sweep

```
## record the delta encoded depth using a plane sweep
deltacovplane = []

## use a list to record the end positions of the elements currently in plane
planelist = []

## BEGIN SWEEP
for r in reads:
    startpos = r[1]
    endpos   = r[2]

    ## clear out any positions from the plane that we have already moved past
    while (len(planelist) > 0):

        if (planelist[0] <= startpos):
            ## the coverage steps down, extract it from the front of the list
            oldend = planelist.pop(0)
            delta
        else:
            break

    ## Now insert the new read into the list
    insertpos = None
    for i in range(len(planelist)):
        if (planelist[i][1] <= startpos < planelist[i+1][1]):
            insertpos = i
            break

    if (insertpos is None):
        planelist.insert(0, endpos)
    else:
        planelist.append(endpos)

    ## Finally record that the coverage has increased
    deltaplane.append((startpos, len(planelist)))

## Flush any remaining end positions
while (len(planelist) > 0):
    oldend = planelist.pop(0)
    deltaplane.append((oldend, len(planelist)))
```

Keep track of end positions of reads that have been seen so far

Check to see if any reads have ended before the start of this one

See notes on how to handle reads that have same coordinates

(Annoying bookkeeping :-/)

Add the end of the current read to the sweep plane in sorted order

Beginning list-based plane sweep over 1873 reads

Plane sweep found 3746 steps, saving 99.62% of the space in 48.90 ms (90.69 speedup)!

Plane Sweep

```
## record the delta encoded depth using a plane sweep
deltacovplane = []

## use a list to record the end positions of the elements currently in plane
planelist = []

## BEGIN SWEEP
for r in reads:
    startpos = r[1]
    endpos = r[2]

    ## clear out any positions from the plane that we have already moved past
    while (len(planelist) > 0):

        if (planelist[0] <= startpos):
            ## the coverage steps down, extract it from the front of the list
            oldend = planelist.pop(0)
            deltacovplane.append((oldend, len(planelist)))
        else:
            break

    ## Now insert the current endpos into the correct position into the list
    insertpos = -1
    for i in xrange(len(planelist)):
        if (endpos < planelist[i]):
            insertpos = i
            break

    if (insertpos > 0):
        planelist.insert(insertpos, endpos)
    else:
        planelist.append(endpos)

    ## Finally record that the coverage has increased
    deltacovplane.append((startpos, len(planelist)))

## Flush any remaining end positions
while (len(planelist) > 0):
    oldend = planelist.pop(0)
    deltacovplane.append((oldend, len(planelist)))
```

Keep track of end positions of reads that have been seen so far

Check to see if any reads have ended before the start of this one

Add the end of the current read to the sweep plane in sorted order

Do we really need the whole list to be sorted?

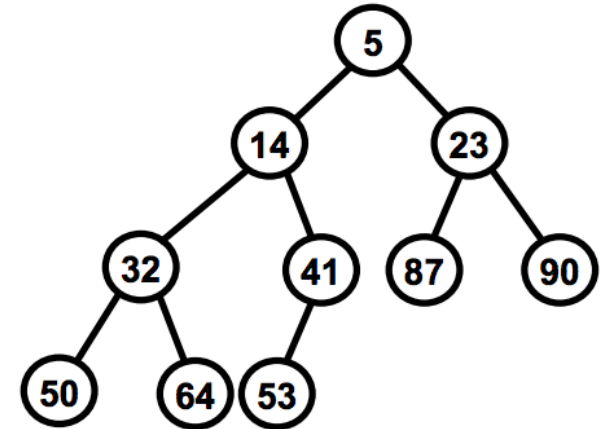
Beginning list-based plane sweep over 1873 reads

Plane sweep found 3746 steps, saving 99.62% of the space in 48.90 ms (90.69 speedup)!

Heaps & Priority Queues

Binary Min Heap: Binary tree such that the value of a node is less than or equal to the value of its 2 children

Similar to a binary search tree, although there are no guarantees about the relationships of the left and right children



Very efficient data structure for dynamically maintaining a set of element while allowing you to find the minimum (or maximum) very fast:

Insert: $O(\lg(n))$	<- super fast
Remove: $O(\lg(n))$	<- super fast
Find-min: $O(1)$	<- instantaneous

Key to fast performance derives from **heap shape property**: the tree is guaranteed to be a complete binary tree, meaning it will remain balanced and the height will always be $\log(n)$

Heaps In Python

8.4. heapq — Heap queue algorithm

Python Software Foundation [US] https://docs.python.org/2/library/heapq.html

Python » 2.7.13 » Documentation » The Python Standard Library » 8. Data Types »

Table Of Contents

- 8.4. heapq — Heap queue algorithm
 - 8.4.1. Basic Examples
 - 8.4.2. Priority Queue Implementation Notes
 - 8.4.3. Theory

Previous topic

8.3. collections — High-performance container datatypes

Next topic

8.5. bisect — Array bisection algorithm

This Page

[Report a Bug](#)
[Show Source](#)

Quick search

Enter search terms or a module, class or function name.

8.4. heapq — Heap queue algorithm

New in version 2.3.

Source code: [Lib/heapq.py](#)

This module provides an implementation of the heap queue algorithm, also known as the priority queue algorithm.

Heaps are binary trees for which every parent node has a value less than or equal to any of its children. This implementation uses arrays for which `heap[k] <= heap[2*k+1]` and `heap[k] <= heap[2*k+2]` for all *k*, counting elements from zero. For the sake of comparison, non-existing elements are considered to be infinite. The interesting property of a heap is that its smallest element is always the root, `heap[0]`.

The API below differs from textbook heap algorithms in two aspects: (a) We use zero-based indexing. This makes the relationship between the index for a node and the indexes for its children slightly less obvious, but is more suitable since Python uses zero-based indexing. (b) Our `pop` method returns the smallest item, not the largest (called a "min heap" in textbooks; a "max heap" is more common in texts because of its suitability for in-place sorting).

These two make it possible to view the heap as a regular Python list without surprises: `heap[0]` is the smallest item, and `heap.sort()` maintains the heap invariant!

To create a heap, use a list initialized to `[]`, or you can transform a populated list into a heap via function `heapify()`.

The following functions are provided:

`heapq.heappush(heap, item)`
Push the value *item* onto the *heap*, maintaining the heap invariant.

`heapq.heappop(heap)`
Pop and return the smallest item from the *heap*, maintaining the heap invariant. If the heap is empty, `IndexError` is raised. To access the smallest item without popping it, use `heap[0]`.

`heapq.heappushpop(heap, item)`
Push *item* on the heap, then pop and return the smallest item from the *heap*. The combined action runs more efficiently than `heappush()` followed by a separate call to `heappop()`.

New in version 2.6.

`heapq.heapify(x)`
Transform list *x* into a heap, in-place, in linear time.

`heapq.heapreplace(heap, item)`
Pop and return the smallest item from the *heap*, and also push the new *item*. The heap size doesn't change. If the heap is empty, `IndexError` is raised.

This one step operation is more efficient than a `heappop()` followed by `heappush()` and can be more appropriate when using a fixed-size heap. The pop/push combination always returns an element from the heap and replaces it with *item*.

The value returned may be larger than the *item* added. If that isn't desired, consider using `heappushpop()` instead. Its push/pop combination returns the smaller of the two values, leaving the larger value on the heap.

The module also offers three general purpose functions based on heaps.

`heapq.merge(*iterables)`
Merge multiple sorted inputs into a single sorted output (for example, merge timestamped entries from multiple log files). Returns an [iterator](#) over the sorted values.

Similar to `sorted(itertools.chain(*iterables))` but returns an iterable, does not pull the data into memory all at once, and assumes that each of the input streams is already sorted (smallest to largest).

New in version 2.6.

`heapq.nlargest(n, iterable[, key])`
Return a list with the *n* largest elements from the dataset defined by *iterable*. *key*, if provided, specifies a function of one argument that is used to extract a comparison key from each element in the *iterable*.

Heap-based Plane-Sweep

```
## record the delta encoded depth using a plane sweep
deltacovplane = []

## use a list to record the end positions of the elements currently in plane
planeheap = []

## BEGIN SWEEP (note change to index based so can peek ahead)
for rr in xrange(len(reads)):
    r = reads[rr]
    startpos = r[1]
    endpos = r[2]

    ## clear out any positions from the plane that we have already moved past
    while (len(planeheap) > 0):

        if (planeheap[0] <= startpos):
            ## the coverage steps down, extract it from the front of the list
            ## oldend = planelist.pop(0)
            oldend = heapq.heappop(planeheap)

            nextend = -1
            if (len(planeheap) > 0):
                nextend = planeheap[0]

            ## only record this transition if it is not the same as a start pos
            ## and only if not the same as the next end point
            if ((oldend != startpos) and (oldend != nextend)):
                deltaxcovplane.append((oldend, len(planeheap)))
            else:
                break

    ## Now insert the current endpos into the correct position into the list
    heapq.heappush(planeheap, endpos)

    ## Finally record that the coverage has increased
    ## But make sure the current read does not start at the same position as the next
    if ((rr == len(reads)-1) or (startpos != reads[rr+1][1])):
        deltaxcovplane.append((startpos, len(planeheap)))

    ## if it is at the same place, it will get reported in the next cycle

## Flush any remaining end positions
while (len(planeheap) > 0):
    ##oldend = planelist.pop(0)
    oldend = heapq.heappop(planeheap)
    deltaxcovplane.append((oldend, len(planeheap)))
```

Heaps in python are built from regular lists

planeheap[0] is min

heapq.heappop() removes from heap

heapq.heappush() adds to heap

Beginning heap-based plane sweep over 1873 reads

Heap-Plane sweep found 3698 steps, saving 99.62% of the space in 14.26 ms (311.08 speedup)!